# PCCST403 OPERATING SYSTEMS

## MODULE 1

**Introduction to Operating Systems**: Operating System Services, Overview of Operating Systems and Kernels, Linux Versus Classic Unix Kernels

**Process concepts:** Process Creation, Process States, Data Structures, Process API, sharing processor among processes - user and kernel modes, context switching, System boot sequence

***Case study**: Linux kernel process management*

**Threads and Concurrency**: Concept of a thread, Multithreading benefits, Multithreading models

***Case study**: The Linux Implementation of Threads*

**Process scheduling**: Concepts and basic algorithms, The Multilevel Feedback Queue: Basic Rules

***Case study**: The Linux Completely Fair Scheduler (CFS), The Linux Scheduling Implementation, Preemption and Context Switching.*

**********

### WHAT IS AN OPERATING SYSTEM?

**An operating system acts as an intermediary between the user of a computer and computer hardware**. The purpose of an operating system is to provide an environment in which a user can execute programs in a convenient and efficient manner.

**Operating System**

- An operating system is a program that controls the execution of application programs. A more common definition is that the operating system is the one program running at all times on the computer (**usually called the kernel),** with all else being system program (ships with the operating system) or application programs.
- An operating system is concerned with the allocation of resources and services, such as memory, processors, devices, and information. The operating system correspondingly includes programs to manage these resources, such as a traffic controller, a scheduler, memory management module, I/O programs, and a file system.
- An operating system is software that manages the computer hardware. The hardware must provide appropriate mechanisms to ensure the correct operation of the computer system and to prevent user programs from interfering with the proper operation of the system

- **OS is a System Software.** Computer system can be divided into four components:

1. **Hardware** – provides basic computing resources example: CPU, memory, I/O devices.
2. **Operating system-**Controls and coordinates use of hardware among various applications and users.
3. **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users. Examples: Word processors, compilers, web browsers, database systems, video games
4. **Users-**People, machines, other computers

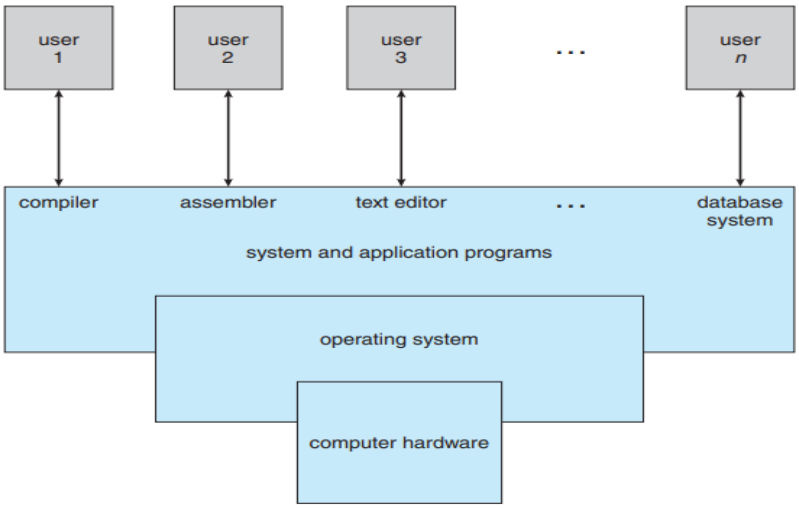



**Figure 1.1 Abstract view of the components of a computer system**

- User View
- Single users: ease of use
- Mainframe or a minicomputer-to maximize resource utilization
- Workstations connected to networks of other workstations and servers.
- System View
- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
  - Controls execution of programs to prevent errors and improper use of the computer.

**FEATURES OF AN OPERATING SYSTEM**

1. **Convenience:** An OS makes a computer more convenient to use.
2. **Efficiency:** An OS allows the computer system resources to be used in an efficient manner.
3. **Ability to Evolve:** An OS should be constructed in such a way as to permit the effective development, testing and introduction of new system functions at the same time without interfering with service.

## OPERATING-SYSTEM OPERATIONS

- Modern operating systems are interrupt driven.
- Events are almost always signaled by the occurrence of an interrupt or a trap.
- A **trap** (or an exception) is a software-generated interrupt caused either by an error (for example, division by zero or invalid memory access) or by a specific request from a user program that an operating-system service be performed.

**1. Dual-Mode and Multimode Operation**

- We need two separate modes of operation: user mode and kernel mode (also called supervisor mode, system mode, or privileged mode). A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).

- With the mode bit, we can distinguish between a task that is executed on behalf of the operating system and one that is executed on behalf of the user.
  - When the computer system is executing on behalf of a user application, the system is in user mode. However, when a user application requests a service from the operating system (via a system call), the system must transition from user to kernel mode to fulfill the request.

**2. Timer**

- We must ensure that the operating system maintains control over the CPU. We cannot allow a user program to get stuck in an infinite loop or to fail to call system services and never return control to the operating system.
- To accomplish this goal, we can use a timer. A timer can be set to interrupt the computer after a specified period.
- The period may be fixed (for example, 1/60 second) or variable (for example, from 1 millisecond to 1 second).
- A variable timer is generally implemented by a fixed-rate clock and a counter.
- The operating system sets the counter.
- Every time the clock ticks, the counter is decremented. When the counter reaches 0, an interrupt occurs.

## BASIC FUNCTIONS OF OPERATING SYSTEM

**The major functions of the operating system are process management, memory management, storage management and I/O system management.**

**1. Process Management:**

- A process is a program in execution. A time-shared user program such as a complier is a process. A word processing program being run by an individual user on a personal computer is a process.
- A system task such as sending output to a printer is also a process. A process needs certain resources including CPU time, memory files & I/O devices to accomplish its task.
- These resources are either given to the process when it is created or allocated to it while it is running. The OS is responsible for the following activities of process management.

1. Creating & deleting both user & system processes.
2. Suspending & resuming processes.
3. Providing mechanism for process synchronization.
4. Providing mechanism for process communication.
5. Providing mechanism for deadlock handling.

**2. Main Memory Management:**

The main memory is central to the operation of a modern computer system. Main memory stores the quickly accessible data shared by the CPU & I/O device. The central processor reads instruction from main memory during instruction fetch cycle & it both reads & writes data from main memory during the

data fetch cycle.

The main memory is generally the only large storage device that the CPU is able to address & access directly. For example, for the CPU to process data from disk; those data must first be transferred to main memory by CPU. Instruction must be in memory for the CPU to execute them. The OS is responsible for the following activities in connection with memory management.

- ✓ Keeping track of which parts of memory are currently being used & by whom.
- ✓ Deciding which processes are to be loaded into memory when memory space becomes available.
- ✓ Allocating & deallocating memory space as needed.

**3. Storage Management**

- ✓ The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the file. The operating system maps files onto physical media and accesses these files via the storage devices

**3.1 File Management:**

- ✓ File management is one of the most important components of an OS. Computer can store information on several different types of physical media - magnetic tape, magnetic disk & optical disk are the most common media. Each medium is controlled by a device such as disk drive or tape drive those has unique characteristics. These characteristics include access speed, capacity, data transfer rate & access method (sequential or random).
- ✓ For convenient use of computer system, the OS provides a uniform logical view of information storage. The OS abstracts from the physical properties of its storage devices to define a logical storage unit - file. A file is collection of related information defined by its creator. The OS is responsible for the following activities of file management.
  - · Creating & deleting files.
  - · Creating & deleting directories.
  - · Supporting primitives for manipulating files & directories.
  - · Mapping files into secondary storage.
  - · Backing up files on non-volatile media.

**3.2 Mass-Storage Management**

**Secondary Storage Management**

The main purpose of computer system is to execute programs. These programs with the data they access must be in main memory during execution. As the main memory is too small to accommodate all data & programs & because the data that it holds are lost when power is lost. The computer system must provide secondary storage to back-up main memory. Most modern computer systems are disks as the storage medium to store data & program. The operating system is responsible for the following activities of disk management.

- ➢ Free space management.
- ➢ Storage allocation.

- Disk scheduling

Because secondary storage is used frequently it must be used efficiently.

### 3.3 Caching

- Caching is an important principle of computer systems. Here's how it works. Information is normally kept in some storage system (such as main memory).
- As it is used, it is copied into a faster storage system—the cache—on a temporary basis. When we need a particular piece of information, we first check whether it is in the cache.
- If it is, we use the information directly from the cache.
- If it is not, we use the information from the source, putting a copy in the cache under the assumption that we will need it again soon.

### 4. I/O System Management:

One of the purposes of an OS is to hide the peculiarities of specific hardware devices from the user. For example, in UNIX the peculiarities of I/O devices are hidden from the bulk of the OS itself by the I/O subsystem. The I/O subsystem consists of:

- A memory management component that includes buffering, catching & spooling.
- A general device- driver interfaces drivers for specific hardware devices. Only the device driver knows the peculiarities of the specific device to which it is assigned.

## OPERATING SYSTEM SERVICES

- **An operating system provides an environment for the execution of programs.**
- **It provides certain services to programs and to the users of those programs.**

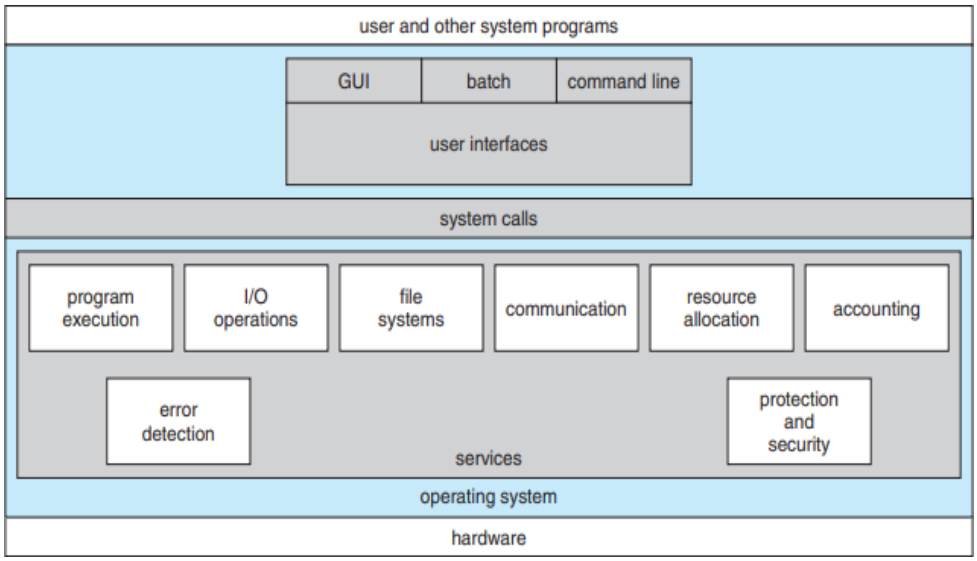



**Fig 1.2 : A view of operating system services**

**Fig 1.3: User Interface**

### 1. User Interface

- Almost all operating systems have a user interface (UI).

- This interface can take several forms.
- One is a command-line interface (CLI), which uses text commands and a method for entering them (say, a keyboard for typing in commands in a specific format with specific options).

❑ Another is a batch interface, in which commands and directives to control those commands are entered into files, and those files are executed. Most commonly, a graphical user interface (GUI) is used.

❑ Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.

**2. Program Execution**

- The system must be able to load a program into memory and to run that program.
- The program must be able to end its execution, either normally or abnormally (indicating error).

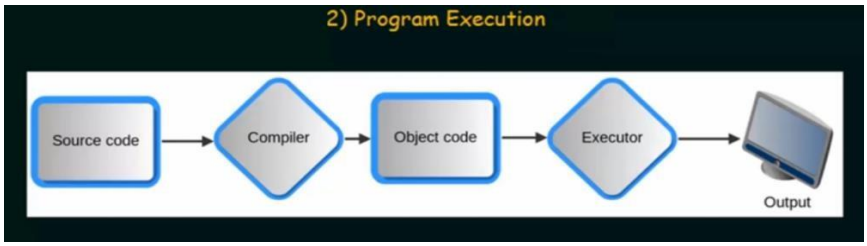



**Fig 1.4: Program Execution**

**3. I/O Operations**

❑ For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide a means to do I/O.

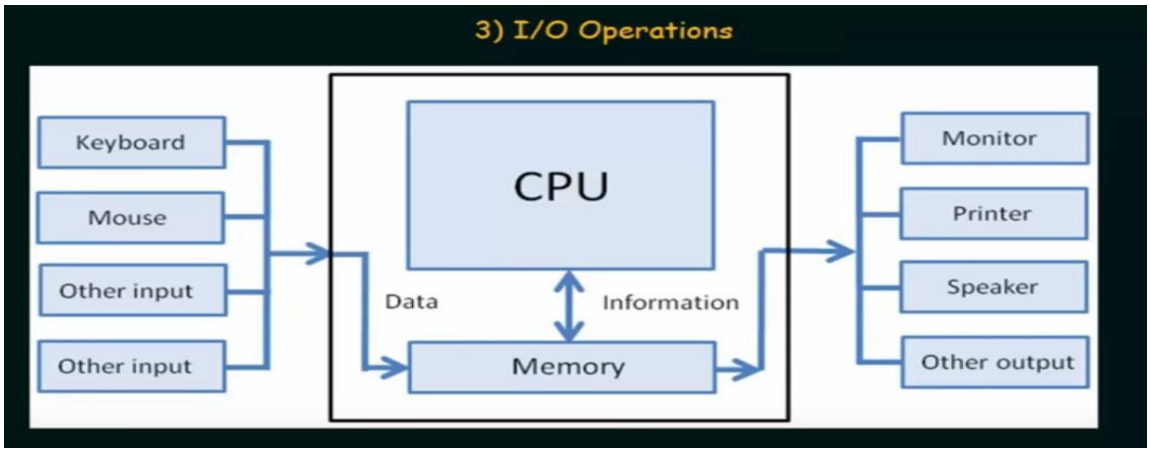



**Fig 1.5: I/O Operations**

**4.File-system manipulation**

Many operating systems provide a variety of file systems, sometimes to allow personal choice and sometimes to provide specific features or performance characteristics.

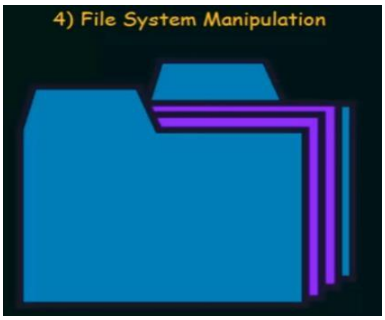



**Fig 1.6: File-system manipulation**

**5. Communications**

- There are many circumstances in which one process needs to exchange information with another process.
- Such communication may occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a computer network.

**Fig 1.7: Communication**

Communications may be implemented via **shared memory**, in which two or more processes read and write to a shared section of memory, or **message passing**, in which packets of information in predefined formats are moved between processes by the operating system.

**6. Error Detection**

- Errors may occur in the CPU and memory hardware (such as a memory error or a power failure), in I/O devices (such as a parity error on disk, a connection failure on a network, or lack of paper in the printer), and in the user program.
- For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing.

**Fig 1.8: Error Detection**

**7. Resource Allocation**

When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them. The operating system manages many different types of resources

For instance, in determining how best to use the CPU, operating systems have CPU-scheduling routines that take into account the speed of the CPU, the jobs that must be executed, the number of registers available, and other factors.

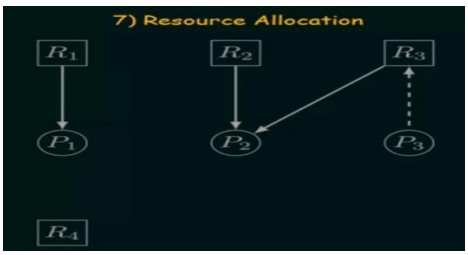



**Fig 1.9: Resource Allocation**

**8. Accounting**

This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.

Usage statistics may be a valuable tool for researchers who wish to reconfigure the system to improve computing services.

**Fig 1.10: Accounting**

### 9. Protection and Security

Protection is any mechanism for controlling the access of processes or users to the resources defined by a computer system. This mechanism must provide means to specify the controls to be imposed and to enforce the controls. It is the job of security to defend a system from external and internal attacks.

**Fig 1.11: Protection and Security**

## SYSTEM CALLS

**System calls provide the interface between a process & the OS**. These are usually available in the form of assembly language instruction. Some systems allow system calls to be made directly from a high level language program like C, BCPL and PERL etc. Systems calls occur in different ways depending on the computer in use.

**Types of System Calls :** There are 5 different categories of system calls –

1. **Process control:** end, abort, create, terminate, allocate and free memory.
2. **File management:** create, open, close, delete, read file etc.
3. **Device management**
4. **Information maintenance**
5. **Communication**

**Services Provided by System Calls :**

- Process creation and management
- Main memory management
- File Access
- Directory and File system management
- Device handling(I/O)
- Protection
- Networking, etc

**How System Call Works? Here are steps for System Call:**

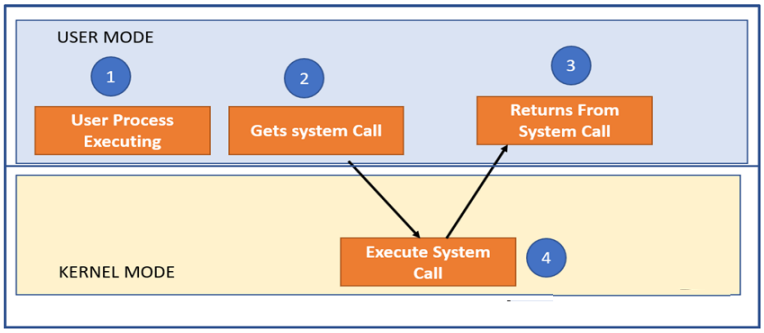



**Fig 1.12 : Working of System Calls**

As you can see in the above-given diagram.
**Step 1)** The processes executed in the user mode till the time a system call interrupts it.
**Step 2)** After that, the system call is executed in the kernel-mode on a priority basis.
**Step 3)** Once system call execution is over, control returns to the user mode.
**Step 4)** The execution of user processes resumed in Kernel mode.

**Why do you need System Calls in OS?**

Following are situations which need system calls in OS:

- **Reading and writing from files demand system calls.**
- **If a file system wants to create or delete files, system calls are required.**
- **System calls are used for the creation and management of new processes.**
- **Network connections need system calls for sending and receiving packets.**
- **Access to hardware devices like scanner, printer, need a system call.**

**1. Process Control:**

- **End, abort:** A running program needs to be able to has its execution either normally (end) or abnormally (abort).
- **Load, execute:** A process or job executing one program may want to load and executes another program.
- **Create Process, terminate process:** There is a system call specifying for the purpose of creating a new process or job (create process or submit job). We may want to terminate a job or process that we created (terminates process, if we find that it is incorrect or no longer needed).
- **Get process attributes, set process attributes:** If we create a new job or process, we should able to control its execution. This control requires the ability to determine & reset the attributes of a job or processes (get process attributes, set process attributes).
- **Wait time:** After creating new jobs or processes, we may need to wait for them to finish their execution (wait time).
- **Wait event, signal event:** We may wait for a specific event to occur (wait event). The jobs or processes then signal when that event has occurred (signal event).
- **Examples**

| | Windows | Unix |
|---|---|---|
| Process Control | CreateProcess()<br>ExitProcess()<br>WaitForSingleObject() | fork()<br>exit()<br>wait() |

**2. File Manipulation:**

· **Create file, delete file:** We first need to be able to create & delete files. Both the system calls

require the name of the file & some of its attributes.

· **Open file, close file:** Once the file is created, we need to open it & use it. We close the file when we are no longer using it.

· **Read, write, reposition file:** After opening, we may also read, write or reposition the file (rewind or skip to the end of the file).

· **Get file attributes, set file attributes:** For either files or directories, we need to be able to determine the values of various attributes & reset them if necessary. Two system calls get file attribute & set file attributes are required for their purpose.

**Examples**

| | Windows | Unix |
|---|---|---|
| File Manipulation | CreateFile()<br>ReadFile()<br>WriteFile()<br>CloseHandle() | open()<br>read()<br>write()<br>close() |

**3. Device Management:**

· **Request device, release device:** If there are multiple users of the system, we first request the device. After we finished with the device, we must release it.

· **Read, write, reposition:** Once the device has been requested & allocated to us, we can read, write & reposition the device.

**Examples**

| | Windows | Unix |
|---|---|---|
| Device Management | SetConsoleMode()<br>ReadConsole()<br>WriteConsole() | ioctl()<br>read()<br>write() |

**4. Information maintenance:**

· **Get time or date, set time or date:** Most systems have a system call to return the current date & time or set the current date & time.

· **Get system data, set system data:** Other system calls may return information about the system like number of current users, version number of OS, amount of free memory etc.

· **Get process attributes, set process attributes:** The OS keeps information about all its processes & there are system calls to access this information.

**Examples**

| | Windows | Unix |
|---|---|---|
| Information Maintenance | GetCurrentProcessID()<br>SetTimer()<br>Sleep() | getpid()<br>alarm()<br>sleep() |

**5. Communication:** There are two modes of communication such as:

**Message passing model:** Information is exchanged through an inter-process communication facility provided by operating system. Each computer in a network has a name by which it is known. Similarly, **each process has a process name which is translated to an equivalent identifier** by which the OS can refer to it. The **get hostid and get processid systems** calls to do this translation. These identifiers are then passed to the general purpose open & close calls provided by the file system or to specific open connection system call. The recipient process must give its permission for communication to take place with an accept connection call. The source of the communication known as client & receiver known as server exchange messages by read message & write message system calls. The close connection call terminates the connection.

**Shared memory model:** Processes use map memory system calls to access regions of memory owned by other processes. They exchange information by reading & writing data in the shared areas. The processes ensure that they are not writing to the same location simultaneously.

**Examples**

| | Windows | Unix |
|---|---|---|
| Communication | CreatePipe()<br>CreateFileMapping()<br>MapViewOfFile() | pipe()<br>shmget()<br>mmap() |

## OVERVIEW OF OPERATING SYSTEMS AND KERNELS

- The user interface is the outer most portion of the operating system, the kernel is the innermost.
- It is the core internals; the software that provides basic services for all other parts of the system, manages hard ware, and distributes system resources.
- The kernel is sometimes referred to as the supervisor, core, or internals of the operating system.
- Typical components of a kernel are interrupt handlers to service interrupt requests, a scheduler to share processor time among multiple processes, a memory management system to manage process address spaces, and system services such as networking and interprocess communication.
- On modern systems with protected memory management units, the kernel typically resides in an elevated system state compared to normal user applications. This includes a protected memory space and full access to the hardware.

- This system state and memory space is collectively referred to as kernel-space. Conversely, user applications execute in user-space. They see a subset of the machine's available resources and can perform certain system functions, directly access hardware, access memory outside of that allotted them by the kernel, or otherwise misbehave.
- When executing kernel code, the system is in kernel-space executing in kernel mode. When running a regular process, the system is in user-space executing in user mode. Applications running on the system communicate with the kernel via system calls (Figure 1.13).
- When an application executes a system call, we say that the kernel is executing on behalf of the application. Furthermore, the application is said to be executing a system call in kernel-space, and the kernel is running in process context. This relationship— that applications call into the kernel via the system call interface— is the fundamental manner in which applications get work done. The kernel also manages the system's hardware. Nearly all architectures, including all systems that Linux supports, provide the concept of interrupts. When hardware wants to communicate with the system, it issues an interrupt that literally interrupts the processor, which in turn interrupts the kernel. A number identifies interrupts and the kernel uses this number to execute a specific interrupt handler to process and respond to the interrupt. In fact, in Linux, we can generalize that each processor is doing exactly one of three things at any given moment:
- In user-space, executing user code in a process
- In kernel-space, in process context, executing on behalf of a specific process
- In kernel-space, in interrupt context, not associated with a process, handling an interrupt

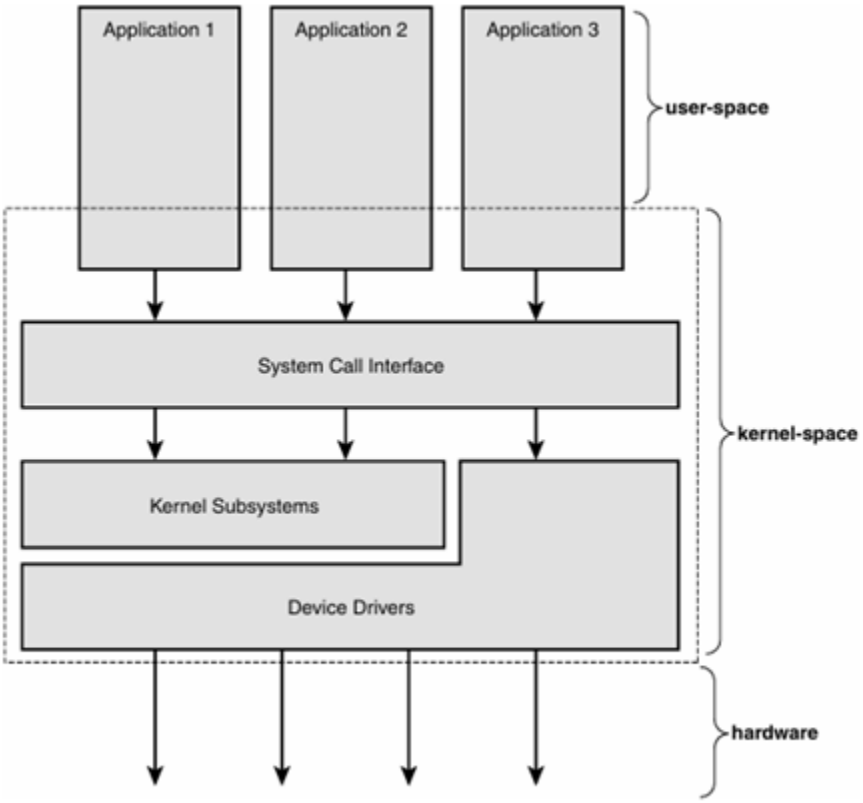



Fig 1.13: Relationship between applications, the kernel, and hardware.

## LINUX VERSUS CLASSIC UNIX KERNELS

- Owing to their common ancestry and same API, modern Unix kernels share various design traits. With few exceptions, a Unix kernel is typically a monolithic static binary. That is, it exists as a single, large, executable image that runs in a single address space. Unix systems typically require a system with a paged memory-management unit (MMU); this hardware enables the system to enforce memory protection and to provide a unique virtual address space to each process.
- Linux historically has required an MMU, but special versions can actually run without one.

### Monolithic Kernel Versus Microkernel Designs

- We can divide kernels into two main schools of design: the monolithic kernel and the micro kernel.
- Monolithic kernels are implemented entirely as a single process running in a single address space. Consequently, such kernels typically exist on disk as single static binaries. All kernel services exist and execute in the large kernel address space. Communication within the kernel is trivial because everything runs in kernel mode in the same address space: The kernel can invoke functions directly, as a user-space application might. Proponents of this model cite the simplicity and performance of the monolithic approach. Most Unix systems are monolithic in design.

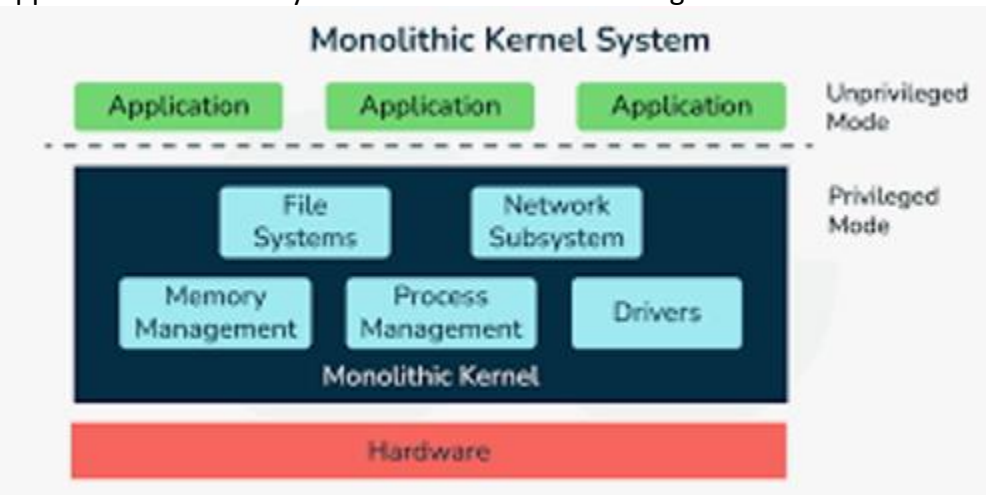



Fig 1.14: Monolithic Kernel

- Microkernels, on the other hand, are not implemented as a single large process. Instead, the functionality of the kernel is broken down into separate processes, usually called servers. Ideally, only the servers absolutely requiring such capabilities run in a privileged execution mode. The rest of the servers run in user-space. All the servers, though, are separated into different address spaces. Therefore, direct function invocation as in monolithic kernels is not possible. Instead, microkernels communicate via message passing: An inter process communication (IPC) mechanism is built into the system, and the various servers communicate with and invoke “services” from each other by sending messages over the IPC mechanism. The separation of the various servers prevents a failure in one server from bringing down another. Likewise, the modularity of the system enables one server to be swapped out for another. Because the IPC mechanism involves quite a bit more overhead than a trivial function call, however, and because a context switch from kernel-space to user-space or vice versa is often involved, message passing includes a latency and throughput hit not seen on monolithic kernels with simple function invocation. Consequently, all

practical microkernel-based systems now place most or all the servers in kernel-space, to remove the overhead of frequent context switches and potentially enable direct function invocation. The Windows NT kernel (on which Windows XP, Vista, and 7 are based) and Mach (on which part of Mac OS X is based) are examples of microkernels. Neither Windows NT nor Mac OS X run any micro kernel servers in user-space in their latest iteration, defeating the primary purpose of micro kernel design altogether.

- The basic idea behind microkernels is to remove all non-essential services from the kernel, and implement them as system applications instead, thereby making the kernel as small and efficient as possible.
- Most microkernels provide basic process and memory management, and message passing between other services, and not much more.
- Security and protection can be enhanced, as most services are performed in user mode, not kernel mode.
- System expansion can also be easier, because it only involves adding more system applications, not rebuilding a new kernel.

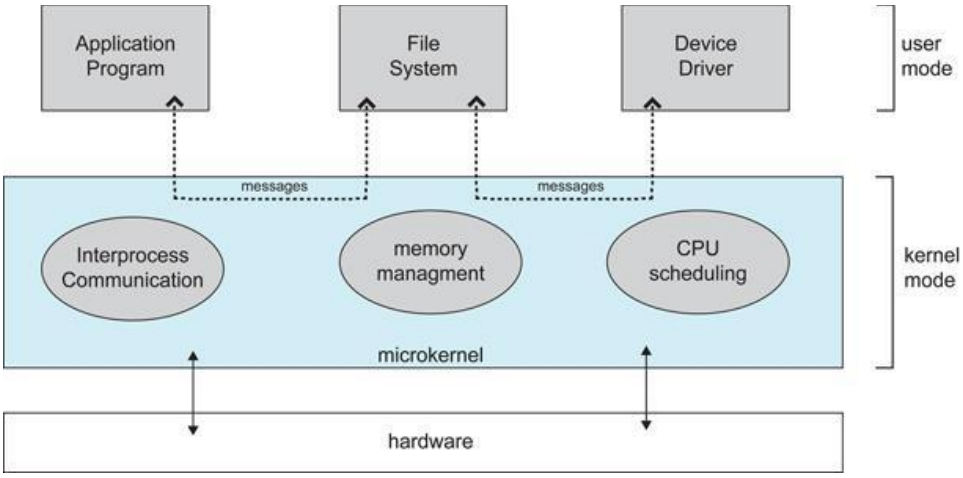



**Fig 1.15: Microkernel structure**

**Functions of microkernel**

- Provide communication between client program and the various services that are also running in the user space.
- Communication is provided through message passing

**Advantages**

- It makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel
- Easier to port from one hardware design to another.
- Microkernel also provides more security and reliability since most services are running as user rather than kernel processes; if a service fails the rest of the OS remains untouched.

Linux is a monolithic kernel; that is, the Linux kernel executes in a single address space entirely in kernel mode. Linux, however, borrows much of the good from microkernels: Linux boasts a modular design, the capability

to preempt itself (called kernel preemption), support for kernel threads, and the capability to dynamically load separate binaries (kernel modules) into the kernel image. Conversely, Linux has none of the performance-sapping features that curse microkernel design: Everything runs in kernel mode, with direct function invocation— not message passing—the modus of communication. Nonetheless, Linux is modular, threaded, and the kernel itself is schedulable.

- As Linus and other kernel developers contribute to the Linux kernel, they decide how best to advance Linux without neglecting its Unix roots (and, more important, the Unix API).
- Linux supports the dynamic loading of kernel modules. Although the Linux kernel is monolithic, it can dynamically load and unload kernel code on demand.
- Linux has symmetrical multiprocessor (SMP) support. Although most commercial variants of Unix now support SMP, most traditional Unix implementations did not.
- The Linux kernel is preemptive. Unlike traditional Unix variants, the Linux kernel can preempt a task even as it executes in the kernel. Of the other commercial Unix implementations, Solaris and IRIX have preemptive kernels, but most Unix kernels are not preemptive.
- Linux takes an interesting approach to thread support: It does not differentiate between threads and normal processes. To the kernel, all processes are the same— some just happen to share resources.
- Linux provides an object-oriented device model with device classes, hot-pluggable events, and a user-space device filesystem (sysfs).
- Linux ignores some common Unix features that the kernel developers consider poorly designed, such as STREAMS, or standards that are impossible to cleanly implement.
- Linux is free in every sense of the word. The feature set Linux implements is the result of the freedom of Linux's open development model.

## Linux Kernel Versions

- Linux kernels come in two flavors: stable and development.
- Stable kernels are production level releases suitable for widespread deployment. New stable kernel versions are released typically only to provide bug fixes or new drivers.
- Development kernels, on the other hand, undergo rapid change where (almost) anything goes.
- As developers experiment with new solutions, the kernel code base changes in often drastic ways. Linux kernels distinguish between stable and development kernels with a simple naming scheme (see Figure 1.16). Three or four numbers, delineated with a dot, represent Linux kernel versions. The first value is the major release, the second is the minor release, and the third is the revision. An optional fourth value is the stable version. The minor release also determines whether the kernel is a stable or development kernel; an even number is stable, whereas an odd number is development. For example, the kernel version 2.6.30.1 designates a stable kernel. This kernel has a major version of two, a minor version of six, a revision of 30, and a stable version of one. The first two values describe the "kernel series".

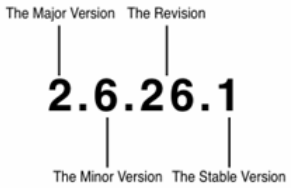


Figure 1.16 Kernel version naming convention.

## PROCESS CONCEPT

**An operating system executes a variety of programs:**

- **batch system** – jobs
- **time-shared systems** – user programs or tasks.

- Process is **a program in execution**, its execution must progress in sequential fashion.
- A program is static and passive whereas a process is dynamic and active.
- **One program can be several processes** (e.g., multiple instances of browser).The process can be started via GUI or command line entry of its name.

## PROCESS STATE

**As a process executes, it changes state**

1. **Start/ New**

This is the initial state when a process is first started/created.

2. Ready

The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run.

3. Running

Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

4. Waiting

Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.

5. Terminated or Exit

Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state.

### Diagram of Process State

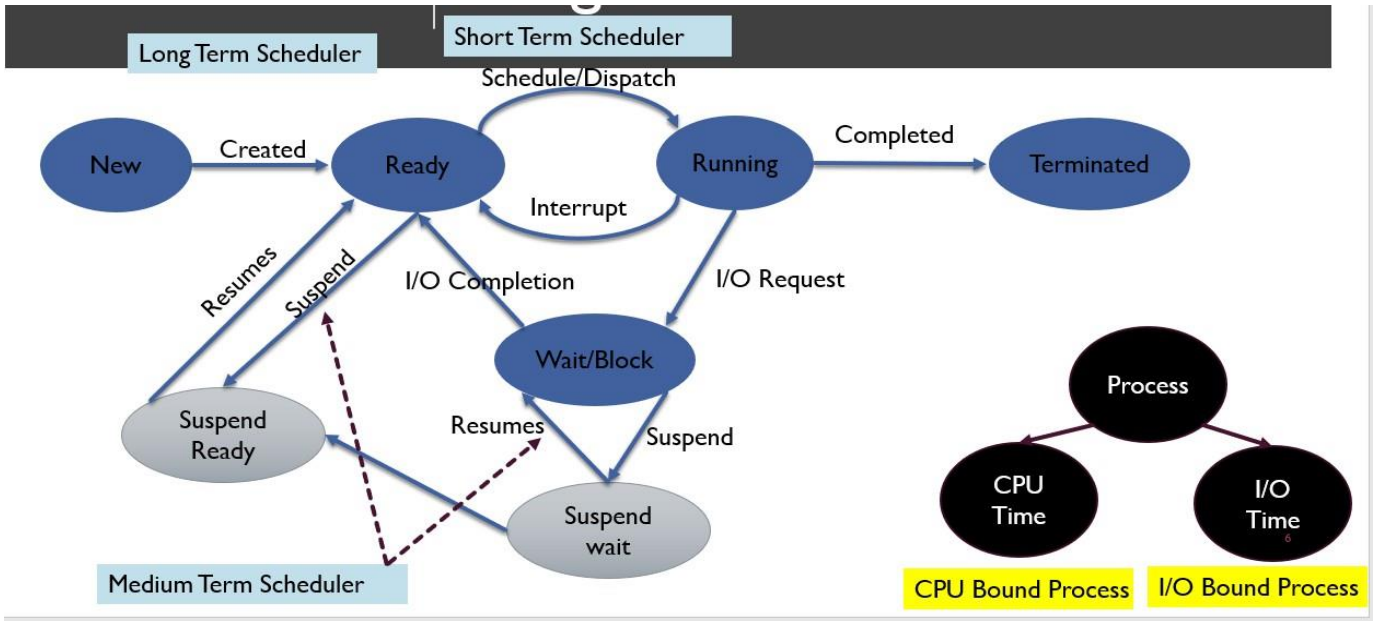



Figure 1.17: Process States and Schedulers

## PROCESS CONTROL BLOCK (PCB)

In the kernel, each process is represented by a **Process Control Block (PCB)**. It is also called a task control block .It contains:

- process number (pid)
- process state
- program counter
- CPU registers
- CPU scheduling information
- memory-management data
- accounting data
- I/O status

**Process state** : The state may be new, ready, running, waiting, terminated, and so on.

**Program counter:** The counter indicates the address of the next instruction to be executed for this process.

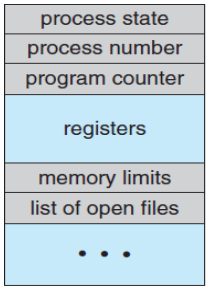
**CPU registers :**The registers vary in number and type, depending on the computer architecture. They include accumulators, index registers, stack pointers, and general-purpose registers, any condition-code information.

**CPU-scheduling information:** This information includes a process priority , pointers to scheduling queues, and any other scheduling parameters.

**Memory-management information :** This information may include such items as the value of the base and limit registers and the page tables, or the segment tables, depending on the memory system used by the operating system.

**Accounting information:** This information includes the amount of CPU and real time used, time limits, job or process numbers, and so on.

**I/O status information:** This information includes the list of I/O devices allocated to the process, a list of open files, and so on.




Process control block (PCB)

Figure 1.18: PCB

**SCHEDULING QUEUES**

- Kernel maintains scheduling queues of processes:
- **job queue**: set of all processes in the system.
- **ready queue**: set of all processes residing in main memory, ready and waiting to execute.
- **device queues**: set of processes waiting for an I/O device.
- Processes migrate among the various queues.
  - For a single-processor system, there will never be more than one running process. If there are more processes, the rest will have to wait until the CPU is free and can be rescheduled.
  - As processes enter the system, they are put into a **job queue,** which consists of all processes in the system.

- The processes that are residing in main memory and are ready and waiting to execute are kept on a list called the **ready queue .**This queue is generally stored as a linked list. A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue . The system also includes other queues. Each device has its own **device queue**.

**Queues for Process Scheduling**

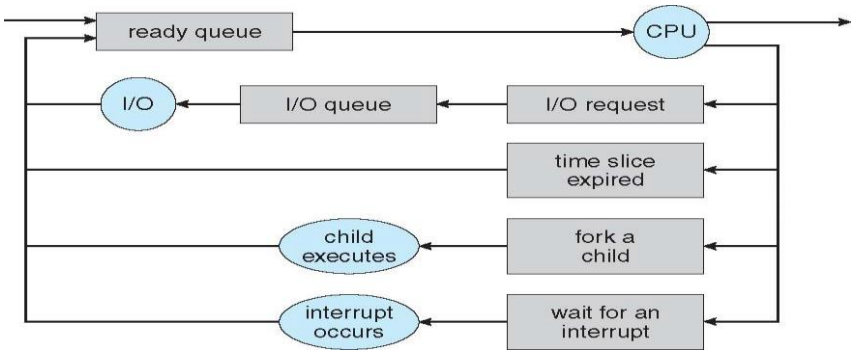



Fig 1.19: Queues for Process Scheduling

A common representation of process scheduling is a **queueing diagram**, such as that in Figure 1.19. Each rectangular box represents a queue. Two types of queues are present: the ready queue and a set of device queues. The circles represent the resources that serve the queues, and the arrows indicate the flow of processes in the system.

A new process is initially put in the ready queue. It waits there until it is selected for execution, or **dispatched**. Once the process is allocated the CPU and is executing, one of several events could occur:

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new child process and wait for the child's termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state and is then put back in the ready queue. A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

## Schedulers

- A process migrates among the various scheduling queues throughout its lifetime.
- The operating system must select, for scheduling purposes, processes from these queues in some fashion. The selection process is carried out by the appropriate **scheduler.**
- In a batch system, more processes are submitted than can be executed immediately. These processes are spooled to a mass-storage device (typically a disk), where they are kept for later execution. The **long-term scheduler, or job scheduler, selects processes from this pool and loads them into memory for** execution.
- The **short-term scheduler, or CPU scheduler, selects from among** the processes that are ready to execute and allocates the CPU to one of them.

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU.Sometimes the only scheduler in a systemShort-term scheduler is invoked frequently (milliseconds)(must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes)
  - (may be slow)
  - The long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***
- **Medium-term scheduler( Mid-term scheduler)-** swap in/out partially executed process to relieve memory pressure. **Medium-term scheduler** can be added if degree of multiple programming needs to decrease

  Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**
  -

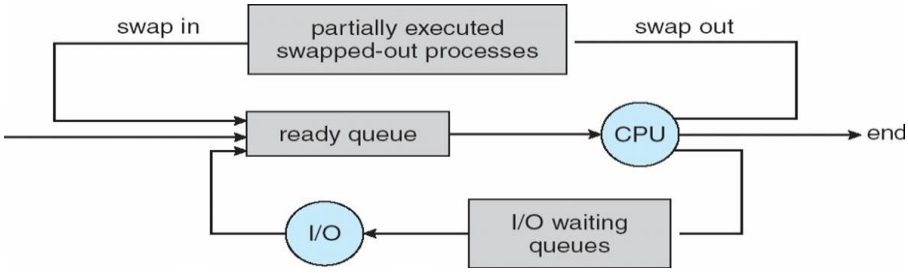



Fig 1.20 : Medium Term Scheduling

## Comparison among Scheduler

| S.N. | Long-Term Scheduler | Short-Term Scheduler | Medium-Term Scheduler |
|---|---|---|---|
| 1 | It is a job scheduler | It is a CPU scheduler | It is a process swapping scheduler. |
| 2 | Speed is lesser than short term scheduler | Speed is fastest among other two | Speed is in between both short and long term scheduler. |
| 3 | It controls the degree of multiprogramming | It provides lesser control over degree of multiprogramming | It reduces the degree of multiprogramming. |
| 4 | It is almost absent or minimal in time sharing system | It is also minimal in time sharing system | It is a part of Time sharing systems. |
| 5 | It selects processes from pool and loads them into memory for execution | It selects those processes which are ready to execute | It can re-introduce the process into memory and execution can be continued. |

## OPERATIONS ON PROCESSES

1. Process Creation
2. Process Termination

### Process Creation

**1.** Process Creation during System initialization

- When an operating system is booted, typically several processes are created.
- Some of these are foreground processes, that is, processes that interact with (human) users and perform work for them.
- Others are background processes, which are not associated with particular users, but instead have some specific function.
- Eg: One background process may be designed to accept incoming e-mail, sleeping most of the day but suddenly get activated when incoming e-mail arrives.
- **Processes that stay in the background** to handle some activity such as e-mail, Web pages, news, printing, and so on are called **daemons.**

**2.** Process Creation by execution of a process creation system call by a running process

Running process will issue system calls to create one or more new processes to help it do its job. **For example**, if a large amount of data is being fetched over a network for subsequent processing, it may be convenient to create one process to fetch the data and put them in a shared buffer while a second process removes the data items and processes them.

**3.** Process Creation by a user request to create a new process

- In interactive systems, users can start a program by typing a command or (double) clicking an icon.

**4.** Process Creation by initiation of a batch job

- Here users can submit batch jobs to the system (possibly remotely).
- When the operating system decides that it has the resources to run another job, it creates a new process and runs the next job from the input queue in it.

During the course of execution, a process may create several new processes.

- The creating process is called a parent process, and the new processes are called the children of that process. Each of these new processes may in turn create other processes, forming a **tree of processes.**
- **Process identifier (or pid),** is associated with each process.
- The **pid** provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.
- Most operating systems (including UNIX, Linux, and Windows) identify processes according to a unique **process identifier** (or **pid**), which is typically an integer number. The pid provides a unique value for each process in the system, and it can be used as an index to access various attributes of a process within the kernel.

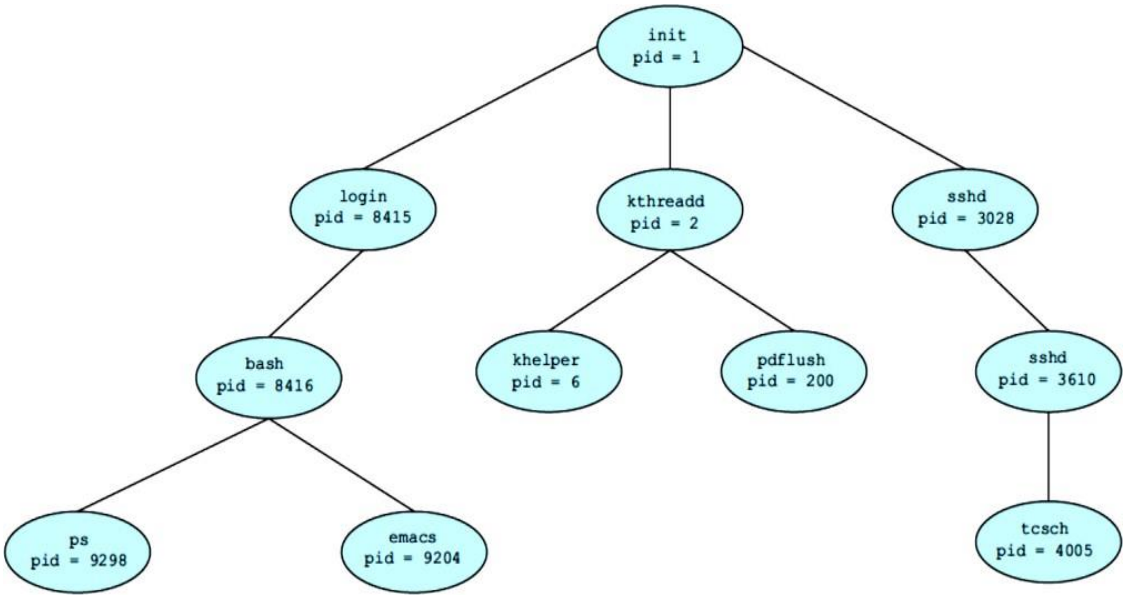


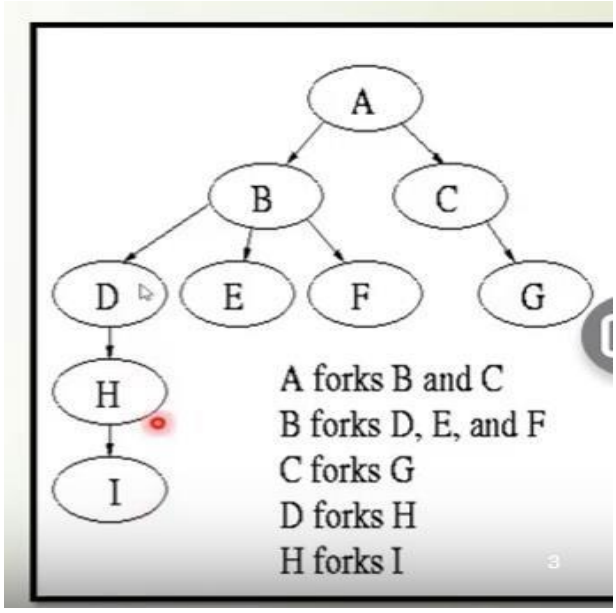

Figure 1.21: Process tree for the Linux operating system

Figure illustrates a typical process tree for the Linux operating system, showing the name of each process and its pid. (We use the term ***process*** rather loosely, as Linux prefers the term ***task*** instead.) The init process (which always has a pid of 1) serves as the root parent process for all user processes. Once the system has booted, the init process can also create various user processes, such as a web or print server, an ssh server, and the like. In Figure 1.22, we see two children of init—kthreadd and sshd. The kthreadd process is responsible for creating additional processes that perform tasks on behalf of the kernel (in this situation, khelper and pdflush). The sshd process is responsible for managing clients that connect to the system by using ssh (which is short for ***secure shell***). The login process is responsible for managing clients that directly log onto the system. In this example, a client has logged on and is using the bash shell, which has been assigned pid 8416. Using the bash command-line interface, this user has created the process ps as well as the emacs editor.

## Resource usage of Child processes

- When a process creates a child process, that child process will need certain resources (CPU time, memory, files, I/O devices) to accomplish its task.
- Resource sharing options:
  1. Parent and children share all resources
  2. Children share subset of parent's resources

3. Parent and child share no resources




Execution of a child process

- When a process creates a new process, two possibilities for execution exist:

**1.** The parent continues to execute concurrently with its children.

**2. The parent waits until some or all of its children have terminated.**

- There are also two address-space possibilities for the new process:

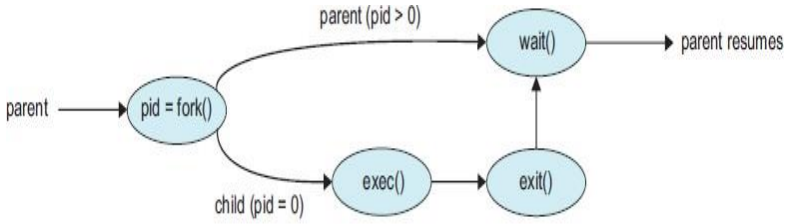
**1. The child process is a duplicate of the parent process** (it has the same program and data as the parent).

**2.** The child process has a new program loaded into it.

**Process Creation Example**

UNIX examples

- **fork()** system call creates new process
- **exec()** system call used after a **fork()** to replace the process' memory space with a new program.
- **wait()** system call issued by a parent process and move itself off the ready queue until the termination of the child




Process creation using the `fork()` system call.

Figure 1.22

## Process Termination

1. Normal exit (voluntary)
   - Most processes terminate because they have done their work.
   - When a compiler has compiled the program given to it, the compiler executes a system call to tell the operating system that it is finished.
   - This call is **exit in UNIX** and **ExitProcess in Windows**.
   - Screen-oriented programs also support voluntary termination.

Eg: Word processors, Internet browsers and similar programs always have an icon or menu item that the user can click to tell the process to remove any temporary files it has open and then terminate.

2. Error exit (voluntary)
   - Error caused by the process, often due to a program bug.
   - Eg: executing an illegal instruction, referencing nonexistent memory, or dividing by zero.
3. Fatal error (involuntary)
   - Process discovers a fatal error.
   - For example, if a user types the command **cc fff.c** to compile the program *fff.c and no such file exists, the compiler simply exits.*
   - Screen-oriented interactive processes generally do not exit when given bad parameters. Instead they pop up a dialog box and ask the user to try again.
4. Killed by another process (involuntary)
   - Process executes a system call telling the operating system to kill some other process.
   - In **UNIX this call is kill.** The corresponding **Win32 function is TerminateProcess**.
   - In both cases, the killer must have the necessary authorization to do in the killee. In some systems, when a process terminates, either voluntarily or otherwise, all processes it created are immediately killed as well.

### Process Termination-System calls used

- A process terminates when it finishes executing its final statement and asks the operating system to delete it by using the **exit()** system call.
- At that point, the process may return a status value (typically an integer) to its parent process (via the wait() system call).
- All the resources of the process like physical and virtual memory, open files, and I/O buffers are deallocated by the operating system.
- A process can cause the termination of another process by system call (for example, TerminateProcess() in Windows).
- Usually, such a system call can be invoked only by the parent of the process that is to be terminated.
- Parent needs to know the identities of its children if it is to terminate them.
- When one process creates a new process, the identity of the newly created process is passed to the parent.

- A parent may terminate the execution of one of its children for a variety of reasons, such as :
- The child has exceeded its usage of some of the resources that it has been allocated.
    - The task assigned to the child is no longer required.
    - The parent is exiting, and the operating system does not allow a child to continue if its parent terminates.
- Some systems do not allow a child to exist if its parent has terminated.
- If a process terminates (either normally or abnormally), then all its children must also be terminated. This phenomenon, referred to as **cascading termination,** is normally initiated by the operating system.
- When a process terminates, its resources are deallocated by the operating system.
- However, its entry in the process table must remain there until the parent calls wait(), because the process table contains the process's exit status.
- A process that has terminated, but whose parent has not yet called wait(), is known as a **zombie** process.
- Once the parent calls wait(), the process identifier of the zombie process and its entry in the process table are released.
- If a parent did not invoke wait() and instead terminated, its child processes left as **orphans.**
- Linux and UNIX handles orphans by assigning the init process as the new parent to orphan processes.
- The init process periodically invokes wait(), thereby allowing the exit status of any orphaned process to be collected and releasing the orphan's process identifier and process-table entry.

## DATA STRUCTURES

The OS is a program, and like any program, it has some key datastructures that track various relevant pieces of information.

To track the state of each process,for example, the OS likely will keep some kind of process list for all processes that are ready, as well as some additional information to track which process is currently running. The OS must also track, in some way, blocked processes; when an I/O event completes, the OS should make sure to wake the correct process and ready it to run again.

**A process has multiple parts:**

- the program **code**, also called **text section**
- runtime **CPU states**, including program counter, registers, etc.
- various types of memory:

  **stack**: temporary data e .g., function parameters, local variables, and return addresses

  **data section**: global variables

  **heap**: memory dynamically allocated during runtime

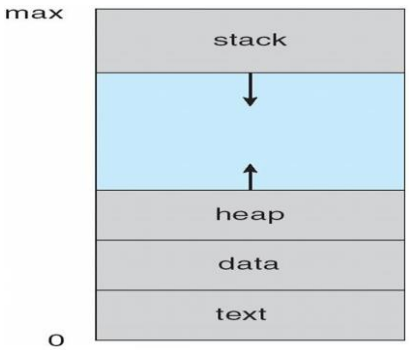



Figure 1.23: Process in Memory

## Process API

- Create: An operating system must include some method to create new processes. When you type a command into the shell, or double-click on an application icon, the OS is invoked to create a new process to run the program you have indicated.
- Destroy: As there is an interface for process creation, systems also provide an interface to destroy processes forcefully. Ofcourse, many processes will run and just exit by themselves when complete; when they don't, however, the user may wish to kill them, and thus an interface to halt a runaway process is quite useful.
- Wait: Sometimes it is useful to wait for a process to stop running; thus some kind of waiting interface is often provided.
- Miscellaneous Control: Other than killing or waiting for a process, there are sometimes other controls that are possible. For example, most operating systems provide some kind of method to suspend a process (stop it from running for a while) and then resume it (continue it running).
- Status: There are usually interfaces to get some status information about a process as well, such as how long it has run for, or what state it is in .

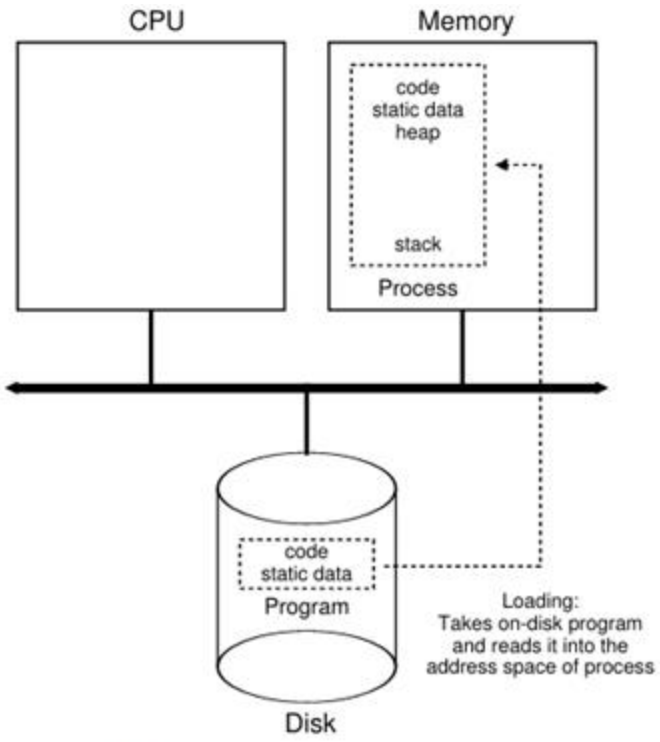



Figure 1.24: **Loading: From Program To Process**

1) fork() SystemCall : The fork system call allows the creation of a new process. When a process calls the fork(), it duplicates itself, resulting in two processes running at the same time. The new process that is created is called a child process. It is a copy of the parent process. The fork system call is required for process creation and enables many important features such as parallel processing, multitasking, and the creation of complex process hierarchies.
2) wait() System Call: A call to wait() blocks the calling process until one of its child processes exits or a signal is

received. After the child process terminates, parent ***continues*** its execution after wait system call instruction.

3) exec() System Call : The exec family of functions is defined in the **unistd.h** header file. The exec functions are used to replace the current running process with a new process. It can be used to run a C program by using another C program.

There are many members in the exec family;one is

**execvp** : Using this command, the created child process does not have to run the same program as the parent process does. The **exec** type system calls allow a process to run any program files, which include a binary executable or a shell script.

## CONTEXT SWITCH

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch.**
- **Context** of a process represented in the PCB. **A context switch** is the process of storing and restoring the state (more specifically, the execution context) of a process so that execution can be resumed from the same point at a later time. This enables multiple processes to share a single CPU and is an essential feature of a multitasking operating system. Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied etc.
- Context-switch times are highly dependent on hardware support. In reality, the real CPU switches back and forth from process to process .This rapid switching back and forth is called **multiprogramming.**
- Context-switch time is overhead; the system does no useful work while switching.
- The more complex the OS and the PCB , the longer the context switch.

**CPU Switch From Process to Process**

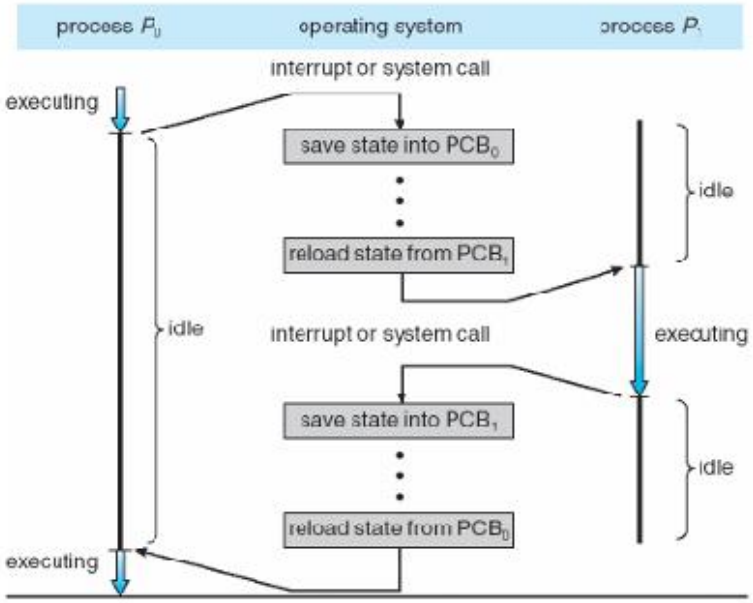



Fig 1.25: **CPU Switch From Process to Process**

# SYSTEM BOOT PROCESS.

**Why is Booting Required ?**

Hardware doesn't know where the operating system resides and how to load it. Need a special program to do this job – **Bootstrap loader.**
**E.g. BIOS – Basic Input Output System.**

Bootstrap loader locates the kernel, loads it into main memory and starts its execution.
In some systems, a simple bootstrap loader fetches a more complex boot program from disk, which in turn loads the kernel.

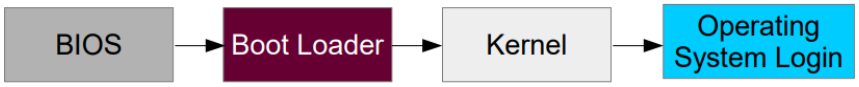



**Fig 1.26: System Booting**

**How Boot process occurs ?**

Reset event on CPU (power up, reboot) causes instruction register to be loaded with a predefined memory location.It contains a jump instruction that transfers execution to the location of Bootstrap program.
This program is form of ROM, since RAM is in unknown state at system startup. ROM is convenient as it needs no initialization and can't be affected by virus.

**System Boot Process.**

BIOS Interaction

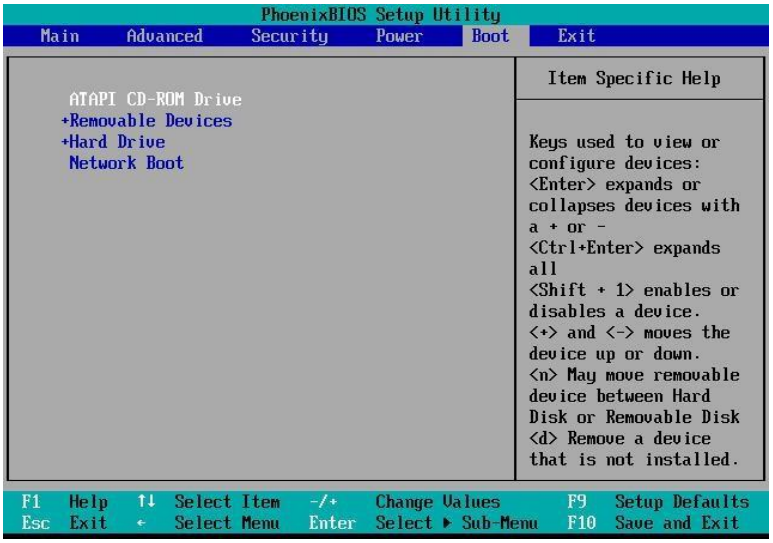



**Fig 1.27 : BIOS Setup**

**Tasks performed at boot up**

- **Run diagnostics** to determine the state of machine. If diagnostics pass, booting continues.

- Runs a **Power-On Self Test** (*POST*) to check the devices that the computer will rely on, are functioning.
- BIOS goes through a preconfigured list of devices until it finds one that is bootable. If it finds no such device, an error is given and the boot process stops.
- Initializes CPU registers, device controllers and contents of the main memory. After this, it loads the OS.
- On finding a bootable device, the BIOS loads and executes its boot sector. In the case of a hard drive, this is referred to as the master boot record (*MBR*) and is often not OS specific.
    - The MBR code checks the partition table for an active partition. If one is found, the MBR code loads that partition's boot sector and executes it.
    - The boot sector is often operating system specific, however in most operating systems its main function is to load and execute a kernel, which continues startup.

**Secondary Boot Loaders**

If there is no active partition or the active partition's boot sector is invalid, the MBR may load a secondary boot loader and pass control to it and this secondary boot loader will select a partition (often via user input) and load its boot sector.
Examples of secondary boot loaders GRUB
– GRand Unified Bootloader LILO
– LInux LOader
NTLDR – NT Loader

## Case study: Linux kernel process management

The Process

A program itself is not a process; a process is an active program and related resources. Indeed, two or more processes can exist that are executing the same program. In fact, two or more processes can exist that share various resources, such as open files or an address space. A process begins its life when it is created. In Linux, this occurs by means of the fork() system call, which creates a new process by duplicating an existing one.The process that calls fork() is the parent, whereas the new process is the child.The parent resumes execution and the child starts execution at the same place: where the call to fork() returns.The fork() system call returns from the kernel twice: once in the parent process and again in the newborn child. Often, immediately after a fork it is desirable to execute a new, different program.The exec() family of function calls creates a new address space and loads a new program into it. In contemporary Linux kernels, fork() is actually implemented via the clone() system call, Finally, a program exits via the exit() system call.This function terminates the process and frees all its resources.A parent process can inquire about the status of a terminated child via the wait() system call, which enables a process to wait for the termination of a specific process.When a process exits, it is placed into a special zombie state that represents terminated processes until the parent calls wait() or waitpid().

**Process Descriptor and the Task Structure**

The kernel stores the list of processes in a circular doubly linked list called the task list. Each element in the task list is a process descriptor of the type struct task_struct, which is defined in .The process descriptor contains all the information about a specific process. The task_struct is a relatively large data structure, at around 1.7 kilobytes on a 32-bit machine.This size, however, is quite small considering that the structure contains all the information that the kernel has and needs about a process.The process descriptor contains the data that describes the executing program—open files, the process's address space, pending signals, the process's state, and much more (Figure 1.28).

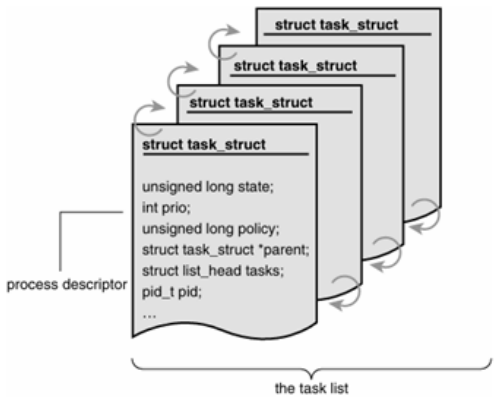



Fig 1.28: The process descriptor and task list.

**Allocating the Process Descriptor**

Prior to the 2.6 kernel series, struct task_struct was stored at the end of the kernel stack of each process.This allowed architectures with few registers, such as x86, to calculate the location of the process descriptor via the stack pointer without using an extra register to store the location.With the process descriptor now dynamically created via the slab allocator, a new structure, struct thread_info, was created that again lives at the bottom of the stack (for stacks that grow down) and at the top of the stack (for stacks that grow up)(Figure 1.29). The thread_info structure is defined on x86 in as

```
struct thread_info {
        struct task_struct      *task;
        struct exec_domain      *exec_domain;
        __u32                   flags;
        __u32                   status;
        __u32                   cpu;
        int                     preempt_count;
        mm_segment_t            addr_limit;
        struct restart_block    restart_block;
        void                    *sysenter_return;
        int                     uaccess_err;
};
```

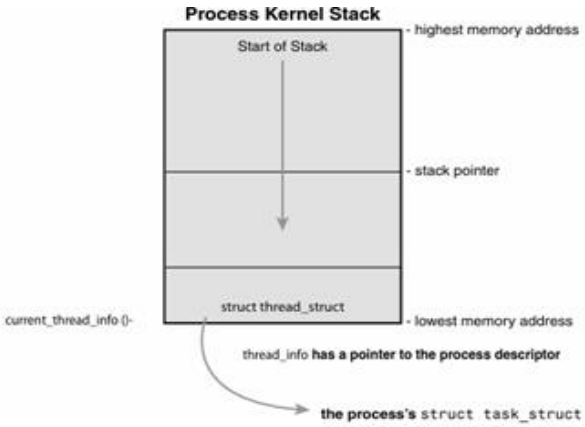



Fig 1.29 The process descriptor and kernel stack.

Each task's thread_info structure is allocated at the end of its stack.The task element of the structure is a pointer to the task's actual task_struct.

**Storing the Process Descriptor**

The system identifies processes by a unique process identification value or PID.The PID is a numerical value represented by pid_t, which is typically an int.The kernel stores this value as pid inside each process descriptor. This maximum value is important because it is essentially the maximum number of processes that may exist concurrently on the system. Inside the kernel, tasks are typically referenced directly by a pointer to their task_struct structure. In fact, most kernel code that deals with processes works directly with struct task_struct. Consequently, it is useful to be able to quickly look up the process descriptor of the currently executing task, which is done via the current macro. This macro must be independently implemented by each architecture. Some architectures save a pointer to the task_struct structure of the currently running process in a register, enabling for efficient access. Other architectures, such as x86 (which has few registers to waste), make use of the fact that struct thread_info is stored on the kernel stack to calculate the location of thread_info and subsequently the task_struct. On x86, current is calculated by masking out the 13 least-significant bits of the stack pointer to obtain the thread_info structure.This is done by the current_thread_info() function.

**Process State**

The state field of the process descriptor describes the current condition of the process .Each process on the system is in exactly one of five different states.This value is represented by one of five flags:

1. TASK_RUNNING—The process is runnable; it is either currently running or on a run queue waiting to run .This is the only possible state for a process executing in user-space; it can also apply to a process in kernel-space that is actively running.
2. TASK_INTERRUPTIBLE—The process is sleeping (that is, it is blocked), waiting for some condition to exist.When this condition exists, the kernel sets the process's state to TASK_RUNNING.The process also awakes prematurely and becomes runnable if it receives a signal.
3. TASK_UNINTERRUPTIBLE—This state is identical to TASK_INTERRUPTIBLE except that it does not wake up and become runnable if it receives a signal.This is used in situations where the process must wait

without interruption or when the event is expected to occur quite quickly. Because the task does not respond to signals in n n this state, TASK_UNINTERRUPTIBLE is less often used than TASK_INTERRUPTIBLE.

4. TASK_TRACED—The process is being traced by another process, such as a debugger, via ptrace.
5. TASK_STOPPED—Process execution has stopped; the task is not running nor is it eligible to run.This occurs if the task receives the SIGSTOP, SIGTSTP, SIGTTIN,or SIGTTOU signal or if it receives any signal while it is being debugged

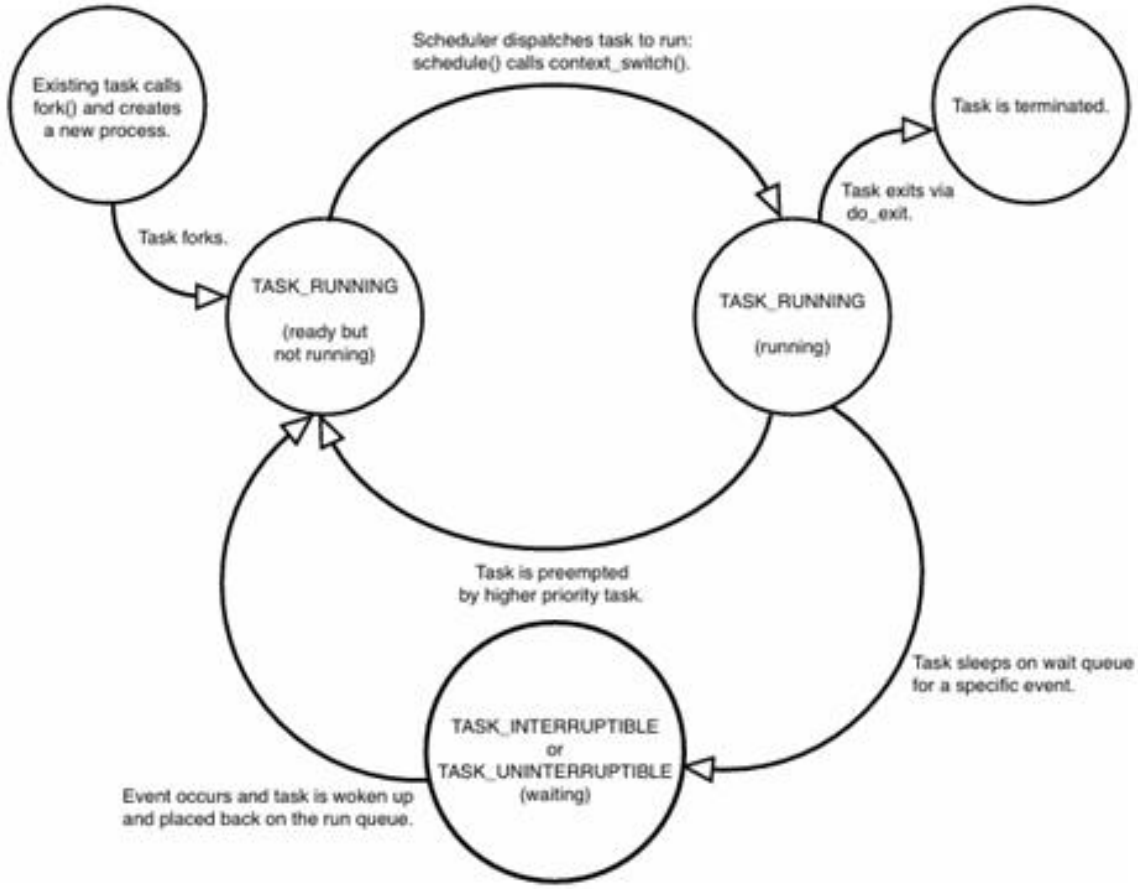



Figure 1.30 Flow chart of process states.

**Manipulating the Current Process State**

Kernel code often needs to change a process's state.The preferred mechanism is using set_task_state(task, state); /* set task 'task' to state 'state' */ This function sets the given task to the given state. If applicable, it also provides a memory barrier to force ordering on other processors. The method set_current_state(state) is synonymous to set_task_state(current, state).

**Process Context**

One of the most important parts of a process is the executing program code.This code is read in from an executable file and executed within the program's address space. Normal program execution occurs in user-space.When a program executes a system call or triggers an exception, it enters kernel-space.At this point, the kernel is

said to be “executing on behalf of the process” and is in process context.When in process context, the current macro is valid. Upon exiting the kernel, the process resumes execution in user-space, unless a higher-priority process has become runnable in the interim, in which case the scheduler is invoked to select the higher priority process. System calls and exception handlers are well-defined interfaces into the kernel.A process can begin executing in kernel-space only through one of these interfaces—all access to the kernel is through these interfaces.

**The Process Family Tree**

A distinct hierarchy exists between processes in Unix systems, and Linux is no exception. All processes are descendants of the init process, whose PID is one.The kernel starts init in the last step of the boot process.The init process, in turn, reads the system initscripts and executes more programs, eventually completing the boot process. Every process on the system has exactly one parent. Likewise, every process has zero or more children. Processes that are all direct children of the same parent are called siblings. The relationship between processes is stored in the process descriptor. Each task_struct has a pointer to the parent’s task_struct, named parent, and a list of children, named children. Consequently, given the current process, it is possible to obtain the process descriptor of its parent with the following code:

**Process Creation**

Process creation in Unix is unique. Most operating systems implement a spawn mechanism to create a new process in a new address space, read in an executable, and begin executing it. Unix takes the unusual approach of separating these steps into two distinct functions: fork()and exec().The first, fork(), creates a child process that is a copy of the current task. It differs from the parent only in its PID (which is unique), its PPID (parent’s PID, which is set to the original process), and certain resources and statistics, such as pending signals, which are not inherited.The second function, exec(), loads a new executable into the address space and begins executing it.The combination of fork()followed by exec()is similar to the single function most operating systems provide.

**Copy-on-Write**

Copy-on-write (or COW) is a technique to delay or altogether prevent copying of the data. Rather than duplicate the process address space, the parent and the child can share a single copy. The data, however, is marked in such a way that if it is written to, a duplicate is made and each process receives a unique copy. Consequently, the duplication of resources occurs only when they are written; until then, they are shared read-only.

**Forking**

Linux implements the fork() via the clone() system call.The clone() system call in turn, calls do_fork()

The bulk of the work in forking is handled by `do_fork()`, which is defined in `kernel/fork.c`. This function calls `copy_process()` and then starts the process running. The interesting work is done by `copy_process()`:

1. It calls `dup_task_struct()`, which creates a new kernel stack, `thread_info` structure, and `task_struct` for the new process. The new values are identical to those of the current task. At this point, the child and parent process descriptors are identical.
2. It then checks that the new child will not exceed the resource limits on the number of processes for the current user.
3. The child needs to differentiate itself from its parent. Various members of the process descriptor are cleared or set to initial values. Members of the process descriptor not inherited are primarily statistically information. The bulk of the values in `task_struct` remain unchanged.
4. The child's state is set to `TASK_UNINTERRUPTIBLE` to ensure that it does not yet run.
5. `copy_process()` calls `copy_flags()` to update the `flags` member of the `task_struct`. The `PF_SUPERPRIV` flag, which denotes whether a task used superuser privileges, is cleared. The `PF_FORKNOEXEC` flag, which denotes a process that has not called `exec()`, is set.
6. It calls `alloc_pid()` to assign an available PID to the new task.
7. Depending on the flags passed to `clone()`, `copy_process()` either duplicates or shares open files, filesystem information, signal handlers, process address space, and namespace. These resources are typically shared between threads in a given process; otherwise they are unique and thus copied here.
8. Finally, `copy_process()` cleans up and returns to the caller a pointer to the new child.

Back in `do_fork()`, if `copy_process()` returns successfully, the new child is woken up and run. Deliberately, the kernel runs the child process first.[8] In the common case of the child simply calling `exec()` immediately, this eliminates any copy-on-write overhead that would occur if the parent ran first and began writing to the address space.

**vfork()**

The vfork()system call has the same effect as fork(), except that the page table entries of the parent process are not copied. Instead, the child executes as the sole thread in the parent's address space, and the parent is blocked until the child either calls exec() or exits. The child is not allowed to write to the address space.

The `vfork()` system call is implemented via a special flag to the `clone()` system call:

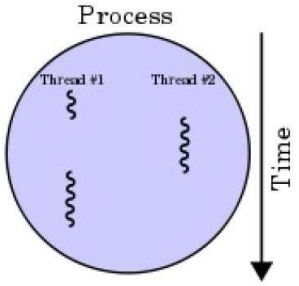
1. In `copy_process()`, the `task_struct` member `vfork_done` is set to `NULL`.
2. In `do_fork()`, if the special flag was given, `vfork_done` is pointed at a specific address.
3. After the child is first run, the parent—instead of returning—waits for the child to signal it through the `vfork_done` pointer.
4. In the `mm_release()` function, which is used when a task exits a memory address space, `vfork_done` is checked to see whether it is `NULL`. If it is not, the parent is signaled.
5. Back in `do_fork()`, the parent wakes up and returns.

## THREADS

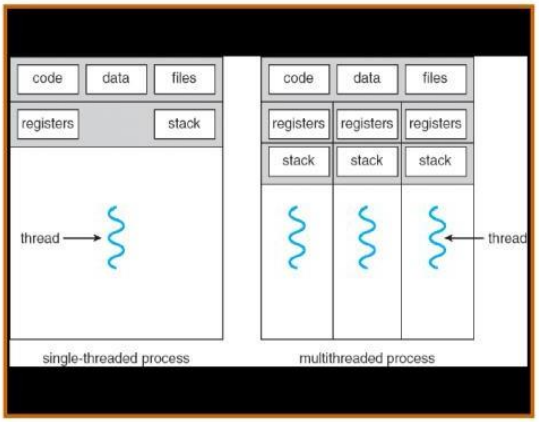
- The single thread of control allows the process to perform only one task at a time. The user cannot simultaneously type in characters and run the spell checker within the same process, for example.
- Most modern operating systems have extended the process concept to allow a process to have multiple threads of execution and thus to perform more than one task at a time. This feature is especially beneficial on multicore systems, where multiple threads can run in parallel. On a system that supports threads, the PCB is expanded to include information for each thread.
- **A thread is a flow of execution through the process code, with its own program counter, system registers and stack(and a thread ID). A thread is also called a light weight process**. Threads provide a way to improve application performance through parallelism. Examples of multithreading include:
- An application that creates photo thumbnails from a collection of images may use a separate thread to generate a thumbnail from each separate image.
- A web browser might have one thread display images or text while another thread retrieves data from the network.

A thread is contained inside a process and different threads in the same process share some resources (most commonly memory), while different processes do not.

Process and threads




SINGLE AND MULTI THREAD



Single Thread
- Has single thread of control
- It allows the process to perform only 1 task at a time.

Multi thread
- Has many threads
- Simultaneous execution of different task

Advantages of Threads :

- Thread minimizes context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- Economy- It is more economical to create and context switch threads.
- Utilization of multiprocessor architectures to a greater scale and efficiency.

Difference between Process and Threads :

| S.N. | Process | Threads |
|---|---|---|
| 1. | Process is heavy weight or resource intensive. | Thread is light weight taking lesser resources than a process. |
| 2. | Process switching needs interaction with operating system. | Thread switching does not need to interact with operating system. |
| 3. | In multiple processing environments each process executes the same code but has its own memory and file resources. | All threads can share same set of open files, child processes. |
| 4. | If one process is blocked then no other process can execute until the first process is unblocked. | While one thread is blocked and waiting, second thread in the same task can run. |
| 5. | Multiple processes without using threads use more resources. | Multiple threaded processes use fewer resources. |
| 6. | In multiple processes each process operates independently of the others. | One thread can read, write or change another thread's data. |

**Benefits**

The benefits of multithreaded programming can be broken down into four major categories:

1. **Responsiveness**. Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. This quality is especially useful in designing user interfaces. For instance, consider what happens when a user clicks a button that results in the performance of a time-consuming operation. A single-threaded application would be unresponsive to the user until the operation had been completed. In contrast, if the time-consuming operation is performed in a separate, asynchronous thread, the application remains responsive to the user.

2. **Resource sharing**. Processes can share resources only through techniques such as shared memory and message passing. Such techniques must be explicitly arranged by the programmer. However, threads share the memory and the resources of the process to which they belong by default. The benefit of sharing code and data is that it allows an application to have several different threads of activity within the same address space.

3. **Economy**. Allocating memory and resources for process creation is costly. Because threads share the resources of the process to which they belong, it is more economical to create and context-switch threads. Empirically gauging the difference in overhead can be difficult, but in general thread creation consumes less time and memory than process creation. Additionally, context switching is typically faster between threads than between processes.

4. **Scalability**. The benefits of multithreading can be even greater in a multiprocessor architecture, where threads may be running in parallel on different processing cores. A single-threaded process can run on only one processor, regardless how many are available.

Disadvantages of Threads :

- Blocking of parent threads will stop all child thread.
- Security.

## Multithreaded Server Architecture

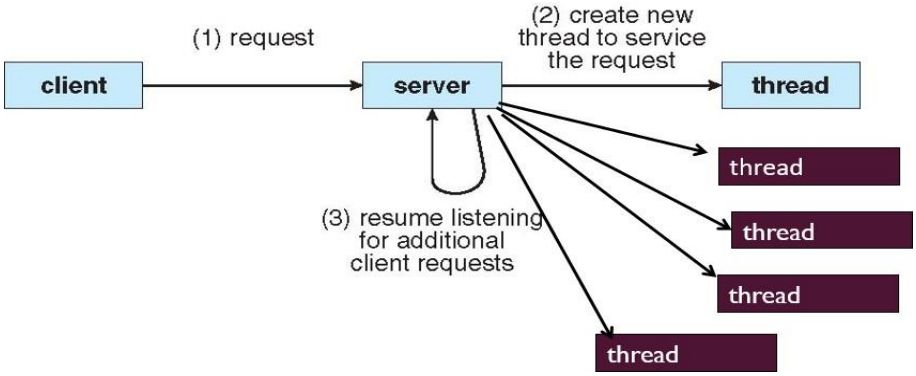



Fig 1.31 : Multithreaded Server Architecture

Types of Threads :

Threads are implemented in following two ways :

- User Level Threads (ULT)
- Kernel Level Threads (KLT)

## User Level Threads (ULT) :

User level thread implement in user level libraries, so thread switching does not need to call operating system and to cause interrupt to the kernel.
In fact, the kernel knows nothing about user level threads and manages them as if they were single-threaded process.

### Advantages of ULT :

- User level threads does not require modification to operating system.
- Easy to represent and manage.
- User level thread can run on any operating system.
- User level threads are fast and efficient.

### Disadvantages of ULT :

- There is a lack of coordination between threads and operating system kernel.
- ULT require non-blocking system call (i.e. Multithreaded kernel)

## Kernel Level Threads (KLT) :

In this method, the kernel knows about and manages the threads. No runtime system is needed in this case. Operating system kernel provides system call to create and manage threads.

### Advantages of KLT :

- Kernel can simultaneously schedule multiple threads from the same process on multiple processes.
- If one thread in a process is blocked the kernel can schedule another thread of the same process.
- Kernel routines themselves can multithreaded.

### Disadvantages of KLT :

- Kernel thread are generally slower to create and manage than the user threads.
- Kernel requires Thread Control Block (TCB) for each thread in the pool, hence complexity increases.

## Difference between ULT and KLT :

| S.N. | User Level Thread | Kernel Level Thread |
|---|---|---|
| 1. | User level thread are faster to create and manage. | Kernel level thread are slower to create and manage. |
| 2. | Implementation is by a thread library at the user level. | Operating system supports creation of kernel level threads. |
| 3. | ULT is generic and can run on any operating system. | KLT is specific to the operating system. |
| 4. | Multi-Threaded can not take advantages of multiprocessing. | Kernel routines themselves can be multithreaded. |

## Multi-Threading Models :

Some operating system provides a combined user level thread and kernel level thread facility. Solaris is a good example of this combined approch.

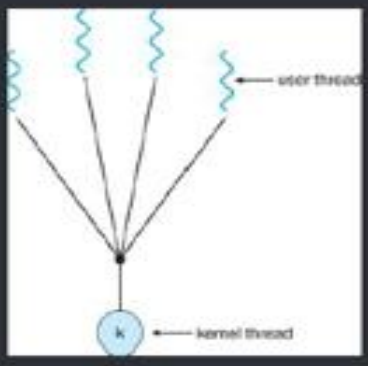
Multi-Threading models are three types.

- Many-to-One Model
- One-to-One Model
- Many-to-Many Model

## Many-to-One Model :

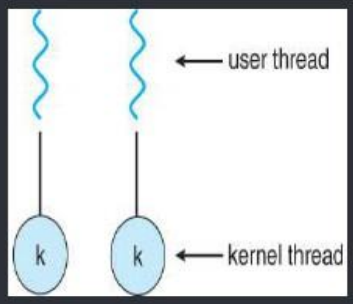
Implementation of the many-to-one model (many user level thread to one kernel level thread) allow the application to create any number of threads that can execute concurrently. In this implementation, all threads activity is restricted to user space.

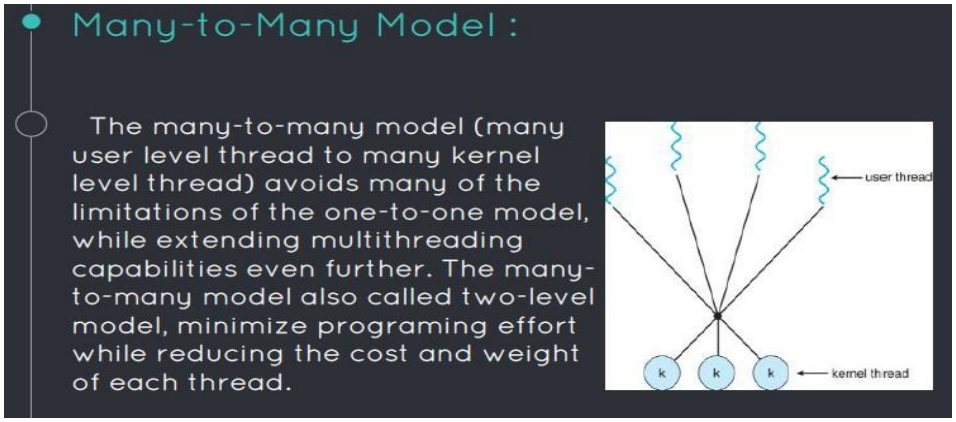
Additionally only one thread can access the kernel at a time, so only one schedulable entity is known to the os.




## One-to-One Model :

The one-to-one model (one user level thread to one kernel level thread) is among the earliest implementations of true multithreading. In this implementation, each ULT created by the application is known to the kernel,and all threads ca access the kernel at the same time.






## Case Study : The Linux Implementation of Threads

To the Linux kernel, there is no concept of a thread. Linux implements all threads as standard processes. The Linux kernel does not provide any special scheduling semantics or data structures to represent threads. Instead, a thread is merely a process that shares certain resources with other processes. Each thread has a unique task_struct and appears to the kernel as a normal process— threads just happen to share resources, such as an address space, with other processes. This approach to threads contrasts greatly with operating systems such as Microsoft Windows or Sun Solaris, which have explicit kernel support for threads (and sometimes call threads lightweight processes).To Linux, threads are simply a manner of sharing resources between processes .The process descriptor describes the shared resources, such as an address space or open files.The threads then describe the resources they alone possess. Conversely, in Linux, there are simply four processes and thus four normal task_struct structures.The four processes are set up to share certain resources.

Creating Threads:Threads are created the same as normal tasks, with the exception that the clone() system call is passed flags corresponding to the specific resources to be shared: clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);

| Flag | Meaning |
|---|---|
| CLONE_FILES | Parent and child share open files. |
| CLONE_FS | Parent and child share filesystem information. |
| CLONE_IDLETASK | Set PID to zero (used only by the idle tasks). |
| CLONE_NEWNS | Create a new namespace for the child. |
| CLONE_PARENT | Child is to have same parent as its parent. |
| CLONE_PTRACE | Continue tracing child. |
| CLONE_SETTID | Write the TID back to user-space. |
| CLONE_SETTLS | Create a new TLS for the child. |
| CLONE_SIGHAND | Parent and child share signal handlers and blocked signals. |
| CLONE_SYSVSEM | Parent and child share System V SEM_UNDO semantics. |
| CLONE_THREAD | Parent and child are in the same thread group. |
| CLONE_VFORK | vfork() was used and the parent will sleep until the child wakes it. |
| CLONE_UNTRACED | Do not let the tracing process force CLONE_PTRACE on the child. |
| CLONE_STOP | Start process in the TASK_STOPPED state. |
| CLONE_SETTLS | Create a new TLS (thread-local storage) for the child. |
| CLONE_CHILD_CLEARTID | Clear the TID in the child. |
| CLONE_CHILD_SETTID | Set the TID in the child. |
| CLONE_PARENT_SETTID | Set the TID in the parent. |
| CLONE_VM | Parent and child share address space. |

**Kernel Threads**

It is often useful for the kernel to perform some operations in the background.The kernel accomplishes this via kernel threads—standard processes that exist solely in kernel space.The significant difference between kernel threads and normal processes is that kernel threads do not have an address space.You can see the kernel threads on your Linux system by running the command ps -ef. Kernel threads are created on system boot by other kernel threads. Indeed, a kernel thread can be created only by another kernel thread.The kernel handles this automatically by forking all new kernel threads off of the kthreadd kernel process.

**Process Termination**

When a process terminates, the kernel releases the resources owned by the process and notifies the child's parent of its demise. Generally, process destruction is self-induced. It occurs when the process calls the exit() system call, either explicitly when it is ready to terminate or implicitly on return from the main subroutine of any program. (That is, the C compiler places a call to exit() after main() returns.) A process can also terminate involuntarily.This occurs when the process receives a signal or exception it cannot handle or ignore.

**Removing the Process Descriptor**

When it is time to finally deallocate the process descriptor, `release_task()` is invoked. It does the following:

1. It calls `__exit_signal()`, which calls `__unhash_process()`, which in turns calls `detach_pid()` to remove the process from the pidhash and remove the process from the task list.
2. `__exit_signal()` releases any remaining resources used by the now dead process and finalizes statistics and bookkeeping.
3. If the task was the last member of a thread group, and the leader is a zombie, then `release_task()` notifies the zombie leader's parent.
4. `release_task()` calls `put_task_struct()` to free the pages containing the process's kernel stack and `thread_info` structure and deallocate the slab cache containing the `task_struct`.

At this point, the process descriptor and all resources belonging solely to the process have been freed.

## PROCESS SCHEDULING(CPU SCHEDULING)

- When a computer is multiprogrammed, it frequently has multiple processes competing for the CPU at the same time. This situation occurs whenever two or more of them are simultaneously in the ready state.
- If only one CPU is available, a choice has to be made which process to run next.
- The part of the operating system that makes the choice is called the scheduler, and the algorithm it uses is called the scheduling algorithm.

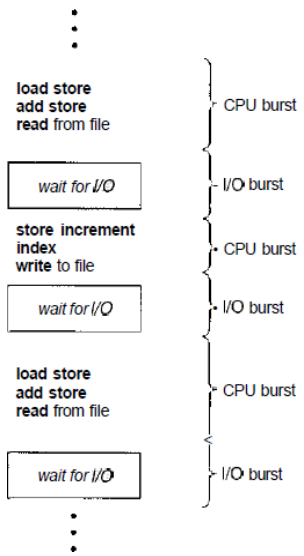
### CPU-I/O Burst Cycle

The success of CPU scheduling depends on an observed property of processes:

- Process execution consists of a **cycle of CPU execution and I/O wait.**

- **Processes** alternate between these two states.
- Process execution begins with a CPU **burst.** That is followed by an **I/O burst, which is followed by another CPU burst, then** another I/O burst, and so on.

Eventually, the final CPU burst ends with a system request to terminate execution




- A process is executed until it must wait, typically for the completion of some I/O request.
- In a simple computer system, the CPU then just sits idle. All this waiting time is wasted; no useful work is accomplished.
- Multiprogramming, use this time productively. Several processes are kept in memory at one time.
- When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process. This pattern continues.
- Every time one process has to wait, another process can take over use of the CPU.

## CPU Scheduler

- Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed.
- The selection process is carried out by the **short-term** scheduler (or CPU scheduler).
- The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of wait for the termination of one of the child processes)
2. When a process switches from the running state to the ready state *(for* example, when an interrupt occurs).
3. When a process switches from the waiting state to the ready state (for example, at completion of I/O).
4. When a process terminates

## Preemptive Scheduling

- A scheduling discipline is pre-emptive if, once a process has been given the CPU ,the CPU can be taken away from that process.
- The scheduling which takes place when a process switches from running state to ready state or from waiting state to ready state is called Preemptive Scheduling.

## Non Pre-emptive Scheduling

- A scheduling discipline is non pre-emptive if, once a process has been given the CPU, the CPU cannot be taken away from that process.
- **Under non-preemptive scheduling, once the CPU has been** allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state

### Dispatcher

- Dispatcher is the module that gives control of the CPU to the process selected by the CPU scheduler.
- This function involves the following:

• Switching context
• Switching to user mode
Jumping to the proper location in the user program to restart that program

### Scheduling Criteria

- Many criteria have been suggested for comparing CPU scheduling algorithms. The criteria include the following:

1. **CPU utilization:** We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).
2. **Throughput:** Number of processes that are completed per time unit, called *throughput. For long processes, this rate may be one* process per hour; for short transactions, it may be 10 processes per second.
3. **Turnaround time:** The interval from the time of submission of a process to the time of completion is the *turnaround time. Turnaround time is the sum of the periods spent waiting* to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.
4. **Waiting time:** Waiting time is the sum of the periods spent waiting in the ready queue.
5. **Response time:** Time from the submission of a request until the first response is produced. This measure, called *response time, is the time it takes to start responding.*

It is desirable **to maximize CPU utilization and throughput** and to **minimize turnaround time, waiting time, and response time**

**Arrival Time-AT Burst Time-BT Completion Time-CT Turnaround Time-TT Waiting Time-WT TT=CT-AT WT=TT-BT**

## SCHEDULING ALGORITHMS

### 1. First-Come, First-Served Scheduling

- Simplest CPU-scheduling algorithm is the **first-come, first-served (FCFS) scheduling algorithm.**
- The process that requests the CPU first is allocated the CPU first.
- The implementation of the FCFS policy is easily managed with a FIFO queue.
- When a process enters the ready queue, its PCB is linked onto the tail of the queue.
- When the CPU is free, it is allocated to the process at the head of the queue.
- The running process is then removed from the queue.
- The average waiting time under the FCFS policy, is often quite long.
- The FCFS scheduling algorithm is non preemptive. Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or

by requesting I/O.

- The FCFS algorithm is troublesome for time-sharing systems, where it is important that each user get a share of the CPU at regular intervals

Convoy effect:

- In a multiprogramming environment, if multiple processes are waiting for the CPU time for execution in "first come first serve" method and a slow process is utilizing the CPU keeping the fast process on wait. It will lead to the convoy effect.
- Consider 100 I/O bound processes and 1 CPU bound job in system.
- These I/O bound processes will quickly pass through the ready queue and suspend themselves (will wait for I/O).Now the CPU-bound process (slow) will get and hold the CPU. During this time, all the other I/O bound processes will finish their I/0 and will move into the ready queue, waiting for the CPU. While the I/O bound processes wait in the ready queue, the I/O devices are idle. Eventually, the CPU-bound process finishes its CPU burst and moves to an I/0 device. All the I/O- bound processes, which have short CPU bursts, execute quickly and move back to the I/0 queues.

At this point, the CPU sits idle. The CPU-bound process will then move back to the ready queue and be allocated the CPU. Again, all the I/O processes end up waiting in the ready queue until the CPU-bound process is done

- There is a convoy effect as all the other processes wait for the one big process to get off the CPU.
- A long CPU-bound job may take the CPU and may force shorter (or I/O-bound) jobs to wait prolonged periods.

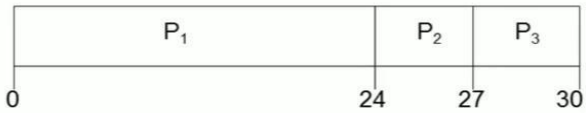
Q1) Find the average waiting time for the following processes using FCFS Scheduling Algorithm

## 1. First-Come, First-Served (FCFS) Scheduling

| Process | Burst Time |
|---|---|
| P1 | 24 |
| P2 | 3 |
| P3 | 3 |

Suppose that the processes arrive in the order: $P_1$ , $P_2$ , $P_3$

- **The Gantt Chart for the schedule is**:

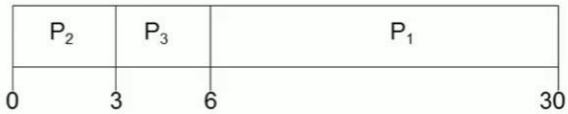



- Waiting time for $P_1$ = 0; $P_2$ = 24; $P_3$ = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

Suppose that the processes arrive in the order:

$P_2, P_3, P_1$

- The Gantt chart for the schedule is:

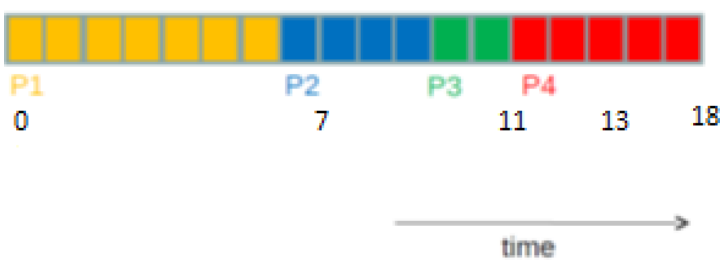



- Waiting time for $P_1 = 6$; $P_2 = 0$, $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case
- *Convoy effect* short process behind long process
- This is Non-Premptive

Q2) Find the average waiting time for the following processes using FCFS Scheduling Algorithm.

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 7 |
| P2 | 0 | 4 |
| P3 | 0 | 2 |
| P4 | 0 | 5 |

Grantt Chart




Average Waiting Time
= (0 + 7 + 11 + 13) / 4
= 7.75

## FCFS Pros and Cons

- Advantages
  - Simple
  - Fair (as long as no process hogs the CPU, every process will eventually run)
- Disadvantages
  - Waiting time depends on arrival order
  - short processes stuck waiting for long process to complete

## 2. Shortest-Job-First (SJF) Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- SJF is optimal – gives minimum average waiting time for a given set of processes
  - The difficulty is knowing the length of the next CPU request

Two schemes:

- **Non-Preemptive:** once CPU given to the process it cannot be preempted until completes its CPU burst.
- **Preemptive:** if a new process arrives with CPU burst length less than remaining time of current executing process, preempt the current process. This scheme is known as the Shortest-remaining-Time-First (SRTF).

**Example #1: Non-reemptive SJF (simultaneous arrival)**

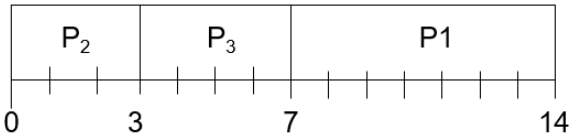
Q1) Find the average waiting time for the following processes using SJF Scheduling Algorithm.

Normal SJF

| Process | Burst Time |
|---|---|
| $P_1$ | 7 |
| $P_2$ | 3 |
| $P_3$ | 4 |

Arrival Time of all processes is 0

The Gantt Chart for SJF (Normal) is:

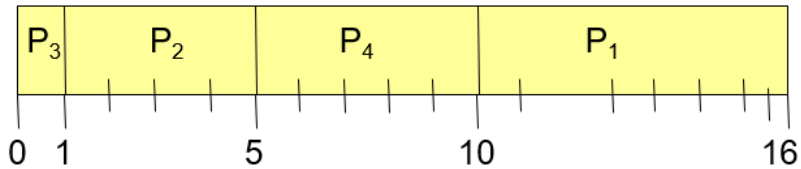



- Average waiting time = (0 + 3 + 7)/3 = 3.33

**Example #2: Non-Preemptive SJF (simultaneous arrival)**

Q2) Find the average waiting time and average turn-around time for the following processes using SJF Scheduling Algorithm.

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0.0 | 6 |
| $P_2$ | 0.0 | 4 |
| $P_3$ | 0.0 | 1 |
| $P_4$ | 0.0 | 5 |



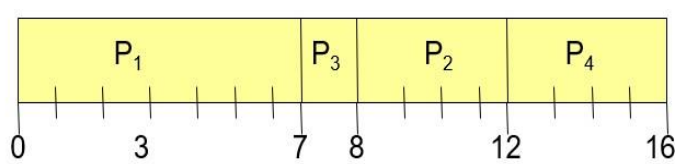

SJF (non-preemptive, simultaneous arrival)

- Average waiting time = (0 + 1 + 5 + 10)/4 = 4
- Average turn-around time = (1 + 5 + 10 + 16)/4 = 8

**Example #3: Non-Preemptive SJF (varied arrival times)**

Q3) Find the average waiting time and average turn-around time for the following processes using SJF Scheduling Algorithm.

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0.0 | 7 |
| $P_2$ | 2.0 | 4 |
| $P_3$ | 4.0 | 1 |
| $P_4$ | 5.0 | 4 |

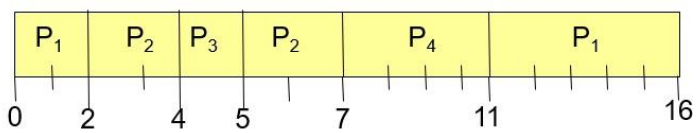



- SJF (non-preemptive, varied arrival times)
- Average waiting time
  = (0 + 6 + 3 + 7)/4 = 4
- Average turn-around time:
  = ( (7 – 0) + (12 – 2) + (8 - 4) + (16 – 5))/4
  = ( 7 + 10 + 4 + 11)/4 = 8

## Example #4: Pre-emptive SJF (Shortest-remaining-time-first)

Q4) Find the average waiting time and average turn-around time for the following processes using SJF Scheduling Algorithm.

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0.0 | 7 |
| $P_2$ | 2.0 | 4 |
| $P_3$ | 4.0 | 1 |
| $P_4$ | 5.0 | 4 |

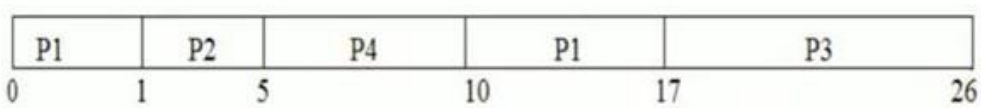



- SJF (preemptive, varied arrival times)
- Average waiting time
  = ( 9 + 1 + 0 + 2)/4
  = 3
- Average turn-around time = [(16 -0)+ (7-2) +( 5 -4)+ (11-5))/4 = 7

## Example #5
## Pre-emptive SJF (Shortest-remaining-time-first)

Q5) Find the average waiting time and average turn-around time for the following processes using SJF Scheduling Algorithm.

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 1 | 4 |
| P3 | 2 | 9 |
| P4 | 3 | 5 |




- The average waiting time: (9+0+15+2)/4 = 26 / 4 = 6.5 msec.
- The average turnaround time: (17+4+24+7) / 4 = 52 / 4 = 13 msec.
  (Completion time – arrival time)

## 3. PRIORITY SCHEDULING

- A priority number (integer) is associated with each process.
- Lager the CPU burst ,lower the priority.
- The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority)
- **Equal priority processes are scheduled in FCFS.** Priorities can be defined either internally or externally.

- Internally defined priorities use some measurable quantity or quantities to compute the priority of a process.
- For example, memory requirements, number of open files , ratio of average I/O burst to average

CPU burst have been used in computing priorities.

- Externally priority is set by criteria that is external to the O.S.; such as importance of the process.
- Priority can be either preemptive or nonpreemptive.
- A preemptive priority will preempt the CPU if the newly arrived process priority is higher than the priority of the currently running process.
- A non preemptive priority will simply put the new highest priority process at the head of the ready

queue.

- Problem :***Starvation*** – *low priority processes may never get the chance to* execute.

Solution : ***Aging*** – *is a technique of gradually, increasing the priority* of the processes that wait in the system for a long time. so eventually the process will become the highest priority and will gain the CPU (i.e., the more time is spending a process in ready queue waiting, its priority becomes higher and higher).

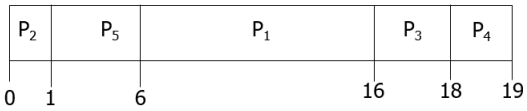
Priority Scheduling Example :

**Non preemptive**

Q1) Find the average waiting time for the following processes using Priority Scheduling Algorithm.

| Process | Burst Time | Priority |
|---|---|---|
| *P1* | 10 | 3 |
| *P2* | 1 | 1 |
| *P3* | 2 | 4 |
| *P4* | 1 | 5 |
| *P5* | 5 | 2 |

- Gantt Chart




- Average waiting time = (6 + 0 + 16 + 18 + 1)/5 = 8.2

Q2)

Consider the below table fo processes with their respective CPU burst times and the priorities.

| PROCESS | BURST TIME | PRIORITY |
|---|---|---|
| P1 | 21 | 2 |
| P2 | 3 | 1 |
| P3 | 6 | 4 |
| P4 | 2 | 3 |

**Draw Gantt chart and find the average waiting time**

The GANTT chart for following processes based on Priority scheduling will be,

| P2 | P1 | P4 | P3 |
|---|---|---|---|

0 3 24 26 32

The average waiting time will be, ( 0 + 3 + 24 + 26 )/4 = 13.25 ms

## 5. ROUND ROBIN SCHEDULING

- The Round Robin is designed for time sharing systems.
- The RR is similar to FCFS, but preemption is added to switch between processes.
- Each process gets a small unit of CPU time (*time quantum or time slice ), usually 10-100 milliseconds. After this time has elapsed,* the process is preempted and added to the end of the ready queue.
- That is, after the time slice is expired an interrupt will occur and a context switch.

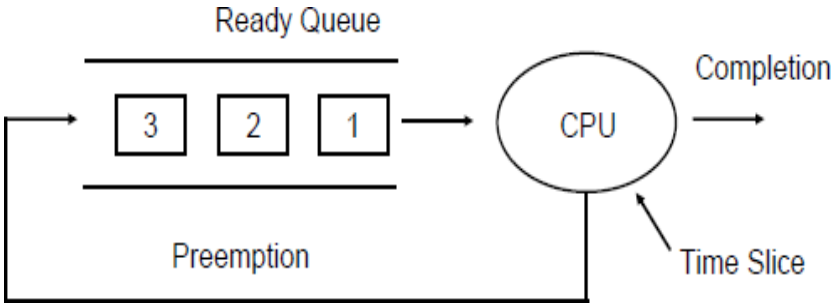



Figure 13: Round Robin Scheduling

- If there are n processes in the ready queue and the time quantum is q, then each process gets 1/n of the CPU time in chunks of at most q time units at once. No process waits more than (n-1)q time units.
- The performance of RR depends on the size of the time slice q :

- If q very large (infinite) ⇒ FIFO
- If q very small ⇒ RR is called processor sharing, and context switch increases. So, q must be large with respect to context switch time, otherwise overhead is too high.

**Q1) Find the average waiting time for the following processes using Round Robin Scheduling Algorithm(Given Quantum time = 4 milliseconds)**

| Process | Burst Time |
|---|---|
| *P1* | 24 |
| *P2* | 3 |
| *P3* | 3 |

- Quantum time = 4 milliseconds
- The Gantt chart is:

| $P_1$ | $P_2$ | $P_3$ | $P_1$ | $P_1$ | $P_1$ | $P_1$ | $P_1$ |
|---|---|---|---|---|---|---|---|

0 4 7 10 14 18 22 26 30

- Average waiting time = {[0+(10-4)]+4+7}/3 = 5.6

**Q2) Find the average waiting time and average turn-around time for the following processes using Round Robin Scheduling Algorithm(Given Quantum time = 20 milliseconds)**

| Process | Burst Time |
|---|---|
| $P_1$ | 53 |
| $P_2$ | 17 |
| $P_3$ | 68 |
| $P_4$ | 24 |

- The Gantt chart is:

| $P_1$ | $P_2$ | $P_3$ | $P_4$ | $P_1$ | $P_3$ | $P_4$ | $P_1$ | $P_3$ | $P_3$ |
|---|---|---|---|---|---|---|---|---|---|

0 20 37 57 77 97 117 121 134 154 162

- Typically, higher average turnaround than SJF, but better *response time*
- Average waiting time
  = ( [(0 – 0) + (77 - 20) + (121 – 97)] + (20 – 0) + [(37 – 0) + (97 - 57) + (134 – 117)] + [(57 – 0) + (117 – 77)] ) / 4
  = (0 + 57 + 24) + 20 + (37 + 40 + 17) + (57 + 40) ) / 4
  = (81 + 20 + 94 + 97)/4
  = 292 / 4 = 73
- Average turn-around time = 134 + 37 + 162 + 121) / 4 = 113.5

## Round Robin Scheduling

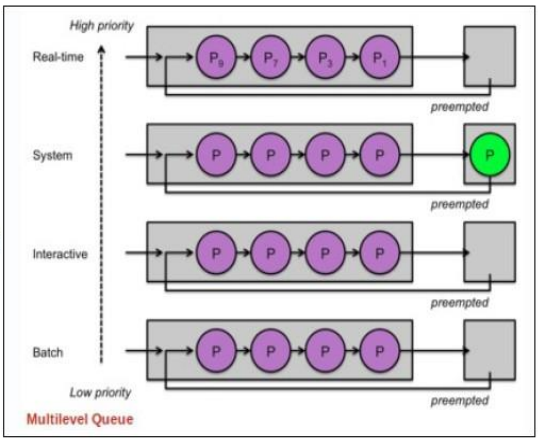
- Advantages
  - Fair (Each process gets a fair chance to run on the CPU)
  - Low average wait time, when burst times vary
  - Faster response time
- Disadvantages
  - Increased context switching

## Priority based Scheduling with large number of processes

- Several processes get assigned the same base priority
  - Scheduling begins to behave more like round robin

# Multilevel Queues

- Processes assigned to a priority classes
- Each class has its own ready queue
- Scheduler picks the highest priority queue (class) which has at least one ready process
- Selection of a process within the class could have its own policy
  - Typically round robin (but can be changed)
  - High priority classes can implement first come first serve in order to ensure quick response time for critical tasks




# More on Multilevel Queues

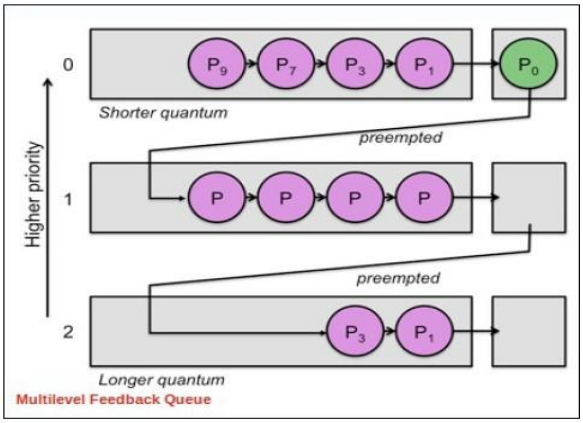
- Scheduler can adjust time slice based on the queue class picked
  - I/O bound process can be assigned to higher priority classes with larger time slice
  - CPU bound processes can be assigned to lower priority classes with shorter time slices
- Disadvantage :
  - Class of a process must be assigned apriori (not the most efficient way to do things!)

## Scheduling: The Multi-Level Feedback Queue

The multi-level feedback queue is an excellent example of a system that learns from the past to predict the future. Such approaches are common in operating systems (and many other places in Computer Science, including hardware branch predictors and caching algorithms). Such approaches work when jobs have phases of behavior and are thus predictable; of course, one must be careful with such techniques, as they can easily be wrong and drive a system to make worse decisions than they would have with no knowledge at all.

### Multilevel feedback Queues (basic Idea)

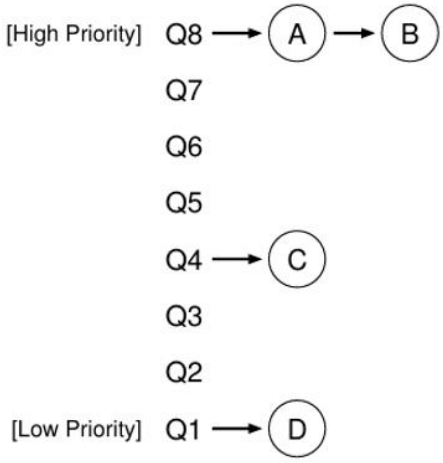
- All processes start in the highest priority class
- If it finishes its time slice (likely CPU bound)
  - Move to the next lower priority class
- If it does not finish its time slice (likely IO bound)
  - Keep it on the same priority class
- As with any other priority based scheduling scheme, starvation needs to be dealt with




MLFQ:Basic Rules

The MLFQ has a number of distinct queues, each assigned a different priority level. At any given time, a job that is ready to run is on a single queue. MLFQ uses priorities to decide which job should run at a given time: a job with higher priority (i.e., a job on a higher queue) is chosen to run. Of course, more than one job may be on a given queue, and thus have the same priority. In this case, we will just use round-robin scheduling among those jobs. Thus, the key to MLFQ scheduling lies in how the scheduler sets priorities. Rather than giving a fixed priority to each job, MLFQ varies the priority of a job based on its observed behavior. If, for example, a job repeatedly relinquishes the CPU while waiting for input from the keyboard, MLFQ will keep its priority high, as this is how an interactive process might behave. If, instead, a job uses the CPU intensively for long periods of time, MLFQ will reduce its priority. In this way, MLFQ will try to learn about processes as they run, and thus use the history of the job to predict its future behavior.

In the figure, two jobs (A and B)are at the highest priority level, while job C is in the middle and Job D is at the lowest priority; the scheduler would just alternate time slices between A and B because they are the highest priority jobs in the system; poor jobs C and D would never even get to run.




- **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't).
- **Rule 2:** If Priority(A) = Priority(B), A & B run in RR.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period $S$, move all the jobs in the system to the topmost queue.

**********END**********